BMAN 71122: Time Series Econometrics

Coursework Assignment

Time Series Analysis of Merck & Co., Inc.
Stock Returns and Spread

The University of Manchester
Alliance Manchester Business School

The University of Manchester — Alliance Manchester Business School

Daily data: January 2000 – December 2025 · Returns · Trading rules · Volatility forecasting

Student Number: 14335120

Abstract

This report examines the time series behaviour of Merck & Co., Inc. (MRK) daily equity data from January 2000 to December 2025 across three exercises. First, the statistical properties of daily log returns are characterised: returns are stationary but strongly non-Gaussian, with excess kurtosis of approximately 28, mild negative skew, and negligible linear autocorrelation at short lags. Second, one-day-ahead rolling-window trading strategies based on ARMA(2,2) and ARMA(2,2)-GARCH(1,1) forecasts are constructed under a strict train/development/test protocol with explicit transaction costs. Neither strategy beats a passive buy-and-hold benchmark out of sample, and the underperformance survives even at zero transaction costs, consistent with weak-form market efficiency. Third, the daily high-low log price spread — a range-based volatility proxy — is forecast using Log-AR(1), Log-HAR, and Log-ARMA(1,2) specifications in a rolling-window design. The Log-HAR and Log-ARMA models significantly outperform the AR(1) benchmark under both MSPE and QLIKE loss (Diebold-Mariano $p < 0.0001$), while being statistically indistinguishable from one another. The overall picture is a familiar asymmetry in financial econometrics: the conditional mean of returns is essentially unpredictable, but conditional volatility is highly forecastable.

Table of Contents

1. Introduction

This report presents an empirical analysis of the time series behaviour of Merck & Co., Inc. (MRK) using daily data spanning January 2000 to December 2025. The objective is twofold: to establish the statistical properties of the return and spread series, and to assess how far those properties can be exploited for prediction — both statistically and, in the case of returns, economically after realistic frictions.

The analysis is organised into three parts. Section 2 characterises the distribution and dependence structure of daily log returns, testing stationarity, normality, and the white-noise hypothesis. Section 3 asks whether any linear predictability in returns is economically meaningful, by building one-day-ahead rolling-window trading rules from ARMA and ARMA-GARCH forecasts and benchmarking them against a moving-average crossover rule and passive buy-and-hold, with explicit transaction costs and formal tests of differential performance. Section 4 turns from the conditional mean to conditional volatility, modelling and forecasting the daily high-low log price spread within a rolling-window framework and comparing Log-HAR and Log-ARMA specifications against a Log-AR(1) benchmark.

The three parts deliver a coherent message. Returns are stationary but fat-tailed and serially uncorrelated at short horizons; the trading rules built on that thin predictability fail to beat passive holding even before costs; yet the volatility proxy is strongly persistent and its one-day-ahead level is forecastable with material accuracy gains over a naive benchmark. This asymmetry — an unpredictable conditional mean alongside a predictable conditional variance — is precisely what the weak-form efficient market hypothesis (Fama, 1970) permits, and it frames the interpretation throughout.

2. Analysis of Stock Returns

2.1 Stationarity

Visual inspection of *Figure 1* shows the log price series exhibiting clear persistence and long swings with no tendency to revert to a fixed mean — the classic signature of a non-stationary, integrated process. The daily log return series, by contrast, fluctuates around a stable mean close to zero with no deterministic trend, suggestive of stationarity. The returns do, however, display occasional large spikes and episodes of elevated volatility, implying that even if returns are stationary in mean, their variance is time-varying — a feature revisited formally in Section 3.

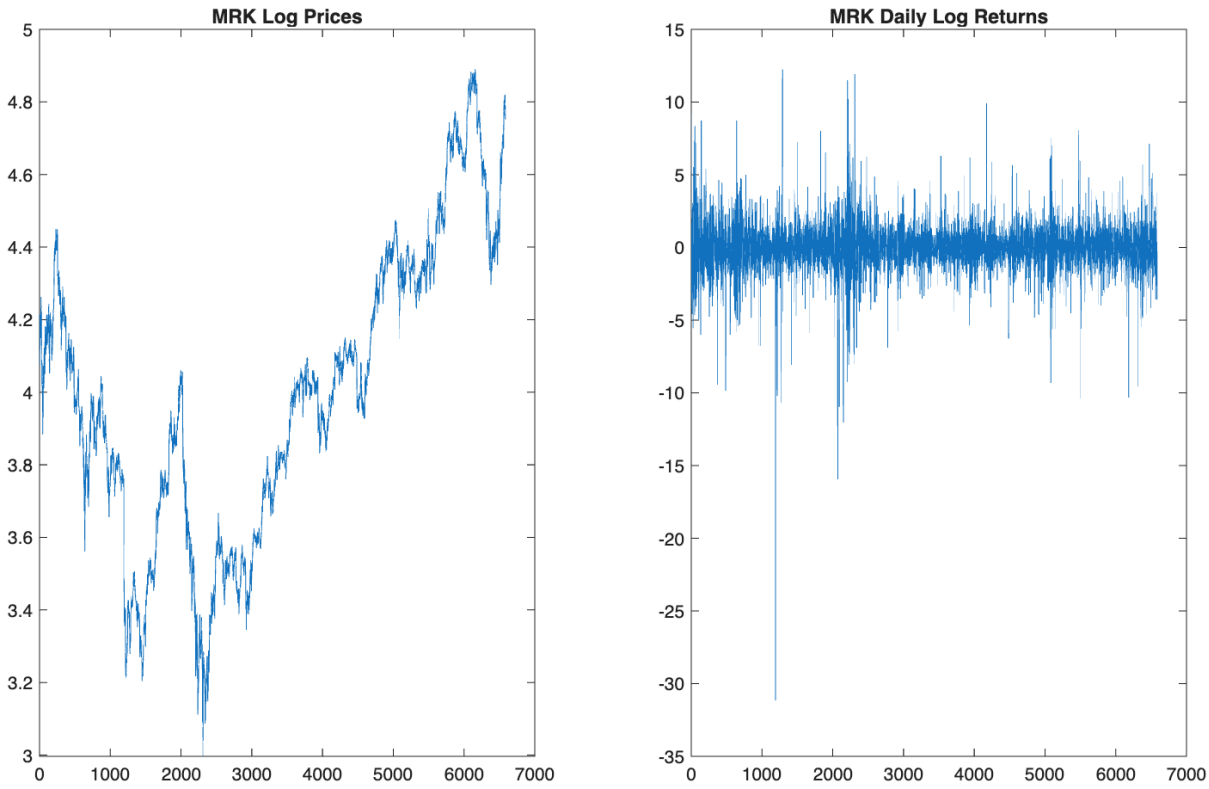

Figure 1: MRK log prices (left) and daily log returns in % (right).

To test formally, we apply the Augmented Dickey-Fuller (ADF) test to both series, with lag length selected by BIC. The null hypothesis is that the series contains a unit root (non-stationary); the alternative is stationarity. Results are reported in *Table 1*.

Series	Test Statistic	p-value	Conclusion
Log Prices	−2.4944	0.3480	Non-Stationary
Log Returns	−34.0532	0.0010	Stationary

Table 1: Augmented Dickey-Fuller test results.

For log prices we fail to reject the unit-root null, consistent with the visual evidence of strong persistence. For log returns the null is rejected decisively, providing strong evidence of stationarity. This pattern is exactly what theory predicts: prices behave like integrated processes, while returns — the first differences of log prices — are far more stable. All subsequent modelling is therefore conducted on returns rather than prices.

2.2 Normality

A long empirical literature documents that stock returns are not Gaussian. Fama (1965), for example, shows that returns exhibit leptokurtosis — more extreme price movements than a normal distribution predicts. We assess this graphically and formally.

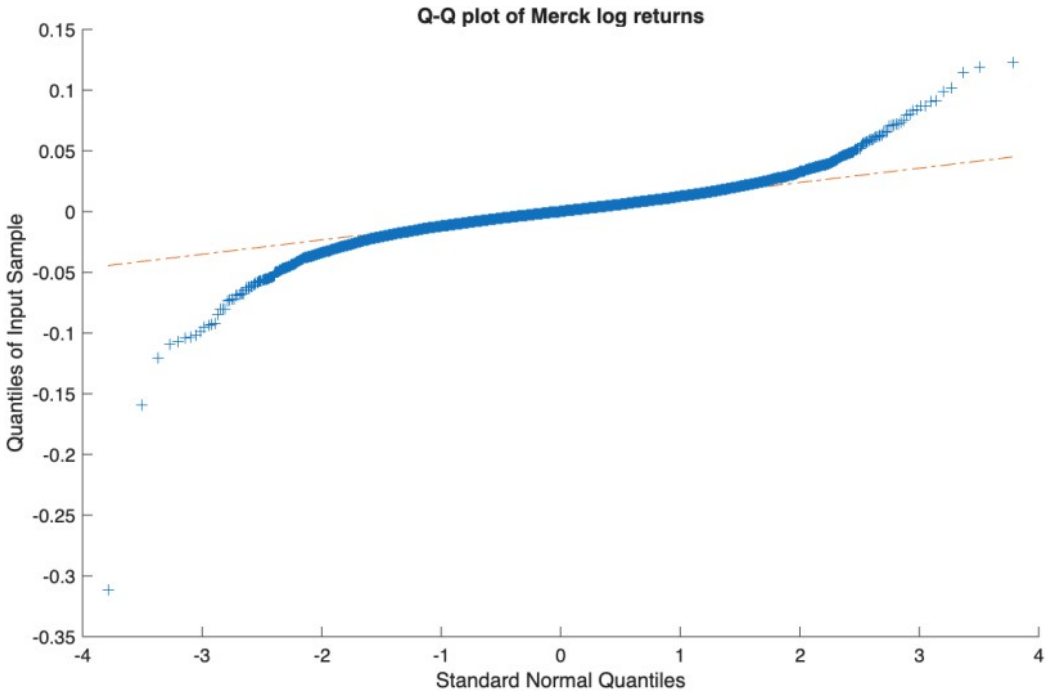

Figure 2: Q-Q plot of MRK daily log returns against a standard normal distribution.

The Q-Q plot departs clearly from the 45-degree line, particularly in both tails, indicating a leptokurtic distribution. The sample kurtosis of approximately 28 — against a Gaussian benchmark of 3 — confirms substantial excess kurtosis, and the sample skewness is negative, as is typical for equity returns.

The Jarque-Bera test (Jarque and Bera, 1987) combines sample skewness and excess kurtosis into a single statistic that is asymptotically $\chi^2(2)$ under the normality null. Results are reported in *Table 2*.

Test	Reject 5%	p-value	Statistic	Critical Value
Jarque-Bera	Yes	0.0010	178581.6742	5.9836

Table 2: Jarque-Bera test for normality of MRK daily log returns.

The statistic exceeds its 5% critical value by more than four orders of magnitude, so normality is rejected overwhelmingly. The unconditional distribution of MRK log returns is not Gaussian — a finding with direct practical consequences, since risk measures and test statistics that assume normality will understate tail risk for this series.

2.3 Skewness and Tail Behaviour

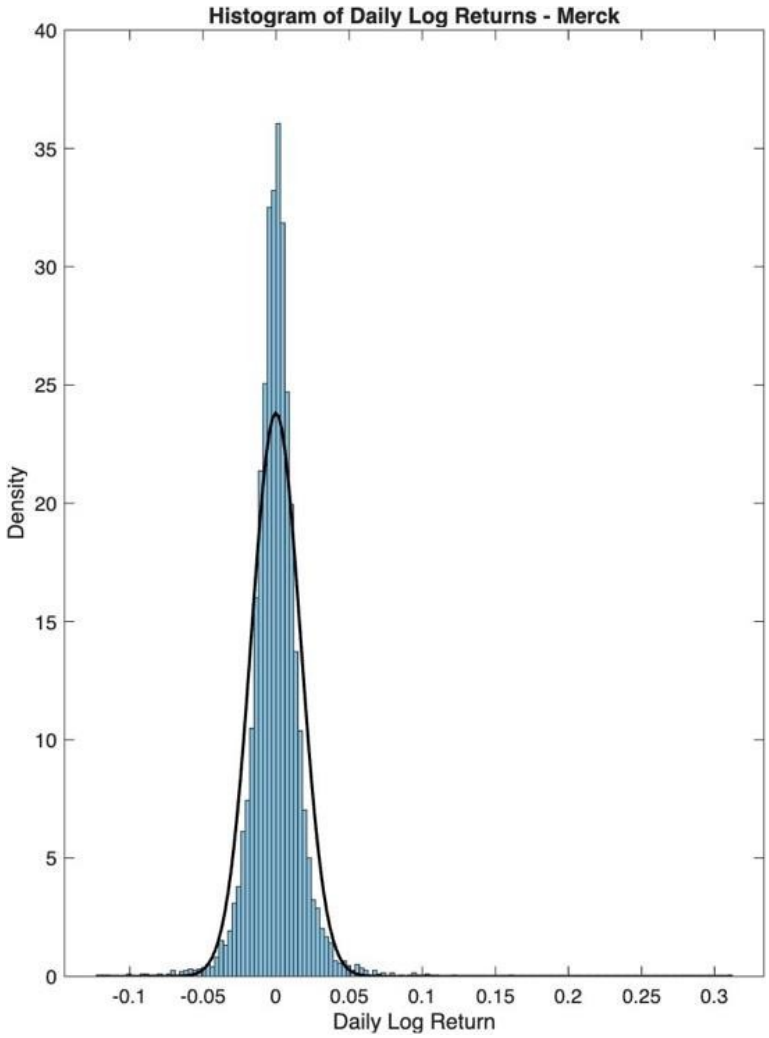

Figure 3: Density histogram of MRK daily log returns with a fitted normal density overlay.

Figure 3 overlays a fitted normal density on the return histogram. The empirical distribution has a far taller and narrower peak than the fitted normal and heavier tails, extending to approximately −0.10 on the left and beyond +0.25 on the right — pronounced leptokurtosis. The left tail appears marginally heavier than the right, consistent with the mild negative skewness measured above. Taken together, the Q-Q plot, the moment estimates and the Jarque-Bera test give a consistent picture: MRK returns deviate strongly from normality, are heavily leptokurtic, and are mildly negatively skewed.

2.4 White Noise and Return Independence

A white noise process has zero mean, constant variance, and zero autocorrelation at all non-zero lags:

$$H_0: \rho(j) = 0 \text{ for all } j \geq 1 \quad vs \quad H_1: \rho(j) \neq 0 \text{ for at least one } j \geq 1$$

Rejection of white noise would imply that log returns are serially correlated, contradicting one of the conditions of weak-form market efficiency. *Figure 4* reports the sample autocorrelation function (ACF) and partial autocorrelation function (PACF). Almost all coefficients lie close to zero and within the 95% confidence bounds, suggesting only weak serial dependence at the individual-lag level.

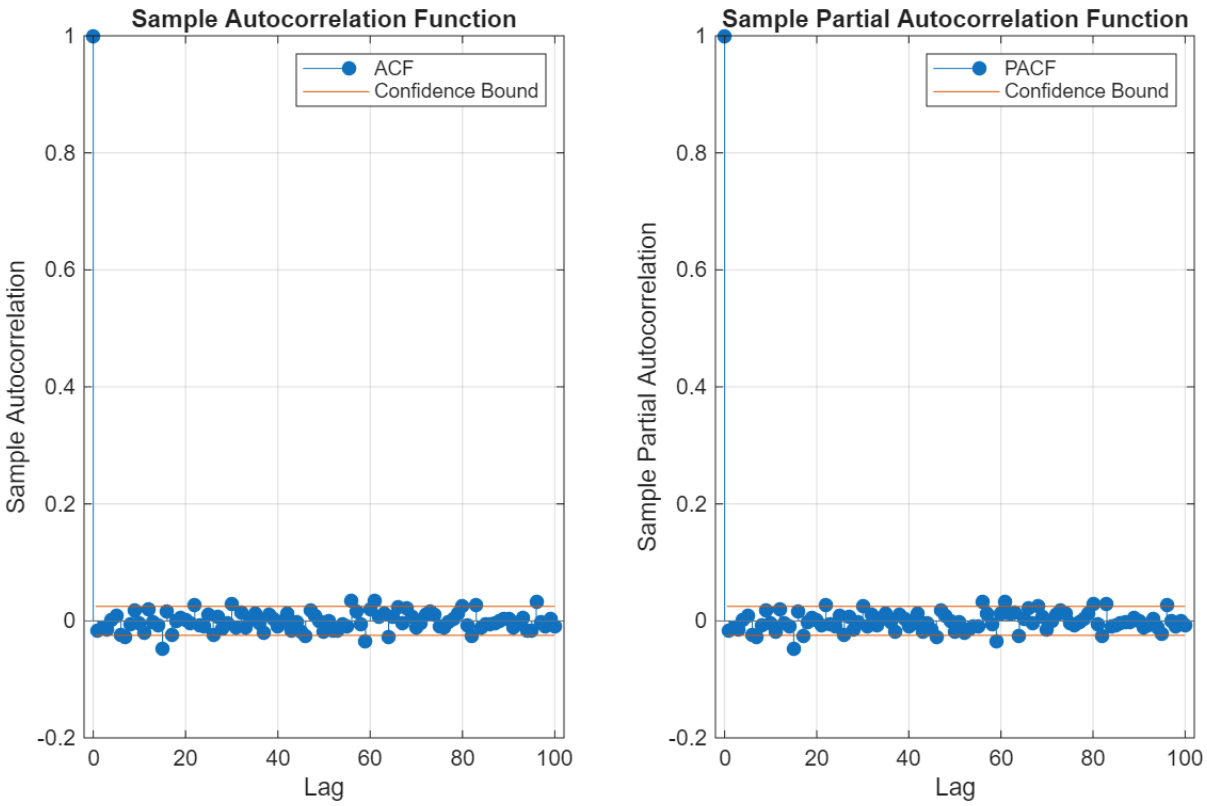

Figure 4: Sample ACF (left) and PACF (right) of MRK daily log returns, lags 1-100.

Beyond testing lags individually, we test the joint significance of multiple lags using the Ljung-Box Q-test (Ljung and Box, 1978), which evaluates whether the first m autocorrelations are jointly zero:

$$H_0: \rho_1 = \rho_2 = \ldots = \rho_m = 0$$

with the alternative that at least one autocorrelation is non-zero, using the statistic:

$$Q(m) = n(n+2) \sum_{k=1}^{m} (n-k)^{-1} r_k^2$$

where n is the sample size and r_k the k-th sample autocorrelation of the centred returns. Under the null, Q(m) is asymptotically $\chi^2(m)$. *Table 3* reports results for lags 1-20.

Lag	Reject 5%	p-value	Q Statistic	Critical Value
1	No	0.1980	1.6573	3.8415
2	No	0.2781	2.5593	5.9915
3	No	0.2603	4.0105	7.8147
4	No	0.4019	4.0308	9.4877
5	No	0.4781	4.5132	11.0700
6	No	0.2328	8.0733	12.5920
7	No	0.0735	12.9440	14.0670
8	No	0.1061	13.1700	15.5070
9	No	0.0796	15.4370	16.9190
10	No	0.1141	15.5230	18.3070
11	No	0.0810	18.0240	19.6750
12	No	0.0550	20.6950	21.0260

Lag	Reject 5%	p-value	Q Statistic	Critical Value
13	No	0.0779	20.7540	22.3620
14	No	0.0966	21.2020	23.6850
15	Yes	0.0018	35.9950	24.9960
16	Yes	0.0016	37.8390	26.2960
17	Yes	0.0008	41.3680	27.5870
18	Yes	0.0014	41.3720	28.8690
19	Yes	0.0020	41.5430	30.1440
20	Yes	0.0032	41.5540	31.4100

Table 3: Ljung-Box Q-test on MRK daily log returns for lags 1-20.

The test fails to reject zero autocorrelation for the first 14 lags, indicating no significant short-term linear serial correlation. From lag 15 onwards, however, the null is rejected, pointing to weak dependence at longer horizons. As a complementary frequency-domain check, the power spectral density is examined in Appendix A.1 and is broadly flat, consistent with a near-white-noise spectrum; variance-ratio tests (Lo and MacKinlay, 1988) offer an alternative time-domain diagnostic for the same hypothesis.

An important distinction closes this section: absence of linear autocorrelation is necessary but not sufficient for independence. A series can be serially uncorrelated in levels yet strongly dependent in higher moments. That is exactly what the volatility clustering visible in Figure 1 suggests, and Section 3 confirms it formally — the Ljung-Box test applied to squared model residuals rejects decisively ($Q = 43.26$, $p < 0.0001$). MRK returns are therefore best described as approximately uncorrelated but not independent: the conditional mean carries little exploitable structure, while the conditional variance carries a great deal. This asymmetry motivates the design of both remaining sections.

3. Rolling-Window ARMA Trading Rule

3.1 Introduction

This section investigates whether the weak dependence documented in Section 2 is economically exploitable. Using MRK daily closing prices from January 2000 to December 2025, we construct one-day-ahead rolling-window trading strategies and evaluate them out of sample. Strategy A applies a plain ARMA specification to generate directional forecasts; Strategy B augments the same mean equation with a GARCH(1,1) conditional variance model, allowing position sizing to adapt to time-varying volatility. Both are benchmarked against a dual moving-average crossover rule (Taylor, 2007, Ch. 7.2.1) and a passive buy-and-hold position. Performance is assessed on cumulative returns, annualised Sharpe ratios, maximum drawdown, and formal statistical tests of differential performance, with explicit accounting for transaction costs. The findings are interpreted with reference to the weak-form efficient market hypothesis (Fama, 1970).

3.2 Methodology

3.2.1 Data, model specification and estimation

MRK daily closing prices are converted into log returns,

$$r_t = \ln P_t - \ln P_{t-1} \qquad (1)$$

giving 6,539 observations split 60/20/20 into training (3,923 obs, Jan 2000-Aug 2015), development (1,307 obs, Aug 2015-Oct 2020) and test (1,309 obs, Oct 2020-Dec 2025) samples. All hyperparameter search takes place on the development sample and the test sample is touched exactly once; separating development from test in this way guards against data snooping (White, 2000). The mean equation is ARMA(p, q),

$$r_t = c + \Sigma_{i=1}^{p} \varphi_i \, r_{t-i} + \Sigma_{j=1}^{\varphi} \theta_j \, \varepsilon_{t-j} + \varepsilon_t \qquad (2)$$

with orders restricted to p, q ∈ {0, 1, 2} for parsimony (Burnham and Anderson, 2002). Model selection uses a three-criterion voting rule across AIC, BIC and HQIC with a Δ = 2 admission band, guarding against the fragility of any single criterion. Residual ARCH effects are diagnosed by a Ljung-Box Q-test on squared residuals (Engle, 1982); where present, a GARCH(1,1) variance equation

$$h_t = \omega + \alpha \, \varepsilon^2_{t-1} + \beta \, h_{t-1} \qquad (3)$$

is appended for Strategy B. Hansen and Lunde (2005) show that GARCH(1,1) is not significantly outperformed by 330 alternative specifications on daily equity data, so higher orders are not pursued. Models are re-estimated every 20 trading days on a rolling window of 3,923 observations, projecting $E[r_{t+1}|F_t]$ and $h_{t+1}|_t$ one step ahead using only past information (Pesaran and Timmermann, 1995). The moving-average benchmark uses a separate 80/20 in-sample/out-of-sample split, as it requires no development sample for hyperparameter search; both approaches share the same out-of-sample test period, with the additional 20% development sample for the ARMA strategies carved from the in-sample portion only.

3.2.2 Signal construction, costs and performance

Each forecast is converted into a discrete position $S_t \in \{-2, -1, 0, +1, +2\}$:

$$S_t = sign(\hat{r}_{t+1}|_t) \cdot size(|\hat{r}_{t+1}|_t|, \tau, k\tau) \qquad (4)$$

A base threshold τ controls entry; positions are doubled when the forecast magnitude exceeds kτ, k ∈ {2, 3}. Three trigger modes are evaluated: raw (threshold applied to the forecast level); volatility-scaled, where the forecast is divided by $\hat{\sigma}_{t+1}|_t$ before comparison so that the trigger is effectively a forecast Sharpe ratio,

$$\hat{r}_{t+1}|_t \, / \, \hat{\sigma}_{t+1}|_t \;\; vs \;\; \tau \qquad (5)$$

and percentile (a rolling 60-day percentile of the absolute forecast). The moving-average benchmark uses the relative gap

$$R_t = (MA\_S(t) - MA\_L(t)) \, / \, MA\_L(t) \qquad (6)$$

with positions of ±1 or 0 relative to a bandwidth B. A 10-day minimum holding period (Brock et al., 1992) suppresses noise-driven reversals across all strategies. Transaction costs are proportional to the absolute position change,

$$TC_t = c \cdot |S_t - S_{t-1}| \qquad (7)$$

with c = 10 basis points, conservatively calibrated for retail execution. The net daily return is $r_t^{net} = S_{t-1} \cdot r_t - TC_t$. Performance is summarised by cumulative and annualised net returns, the annualised Sharpe ratio against a 4% risk-free rate, maximum drawdown, and forecast hit rate. Differences in mean returns are tested with a Newey-West (1987) HAC t-statistic and differences in Sharpe ratios with the HAC-

corrected Jobson-Korkie test of Ledoit and Wolf (2008), both computed on the overlapping test window.

3.3 Empirical Results

3.3.1 ARMA order selection and GARCH diagnostics

Table 4 reports information criteria for the candidate ARMA(p, q) specifications. ARMA(2, 2) is preferred by all three criteria simultaneously and is the only model retained: its AIC is 30.9 units below ARMA(1, 1), far outside the $\Delta = 2$ admission band. ARMA(2, 2) is therefore carried forward as the mean equation for both strategies.

ARMA(p, q)	AIC	BIC	HQIC	Parameters
ARMA (0, 1)	−20,318.29	−20,299.46	−20,311.61	3
ARMA (0, 2)	−20,320.94	−20,295.85	−20,312.04	4
ARMA (1, 0)	−20,318.28	−20,299.46	−20,311.60	3
ARMA (1, 1)	−20,321.82	−20,296.73	−20,312.92	4
ARMA (1, 2)	−20,319.22	−20,287.85	−20,308.09	5
ARMA (2, 0)	−20,321.10	−20,296.00	−20,312.19	4
ARMA (2, 1)	−20,319.35	−20,287.98	−20,308.22	5
ARMA (2, 2)	−20,352.72	−20,315.07	−20,339.36	6

Table 4: Information criteria for candidate ARMA(p, q) models, MRK training sample (3,923 observations). ARMA(2, 2) minimises all three criteria simultaneously.

The Ljung-Box Q-statistic on squared ARMA(2, 2) residuals is 43.26 ($p < 0.0001$), decisively rejecting the no-ARCH null and motivating GARCH(1,1) for Strategy B. Estimated variance parameters are $\omega = 1.8 \times 10^{-5}$, $\alpha = 0.0521$ and $\beta = 0.8924$, giving persistence $\alpha + \beta = 0.9445$ — covariance-stationary, with a shock half-life of roughly twelve trading days. After GARCH fitting, the Ljung-Box statistic on squared standardised residuals falls to 0.16 ($p \approx 1.00$), confirming that the variance specification is adequate. The training-sample mean conditional standard deviation, $\sigma^* = 0.01738$, is used as the volatility target for position scaling.

3.3.2 Development-sample selection of trading rules

Table 5 reports the top configurations per signal mode. For Strategy A, the raw-threshold rule with $\tau = 1 \times 10^{-4}$ and $k = 3$ dominates at a development Sharpe of 0.3961, versus 0.2440 (percentile) and 0.0621 (volatility-scaled): raw forecast levels carry more information than their volatility-adjusted equivalents for this series. Strategy B's best configuration is raw with $\tau = 10 \times 10^{-4}$, $k = 2$, at a Sharpe of 0.2601 on just 65 trades. The development buy-and-hold Sharpe is 0.2093.

Strategy	Mode	Threshold	DD Mult	Net Sharpe	Trades	Flat %
A: ARMA(2,2) raw, k=3	raw	0.0001	3	0.3961	418	26.2
A: ARMA(2,2) raw, k=2	raw	0.0001	2	0.3357	406	26.2
A: ARMA(2,2) percentile	percentile	0.75	1	0.2440	116	55.2
A: ARMA(2,2) raw, single	raw	0.0001	1	0.2428	178	26.2
A: ARMA(2,2) vol-scaled	vol-scaled	0.005	3	0.2086	460	21.5
B: ARMA-GARCH raw, k=2	raw	0.001	2	0.2601	65	76.7

Strategy	Mode	Threshold	DD Mult	Net Sharpe	Trades	Flat %
B: ARMA-GARCH raw, single	raw	0.001	1	0.1973	59	76.7
B: ARMA-GARCH raw, k=3	raw	0.001	3	0.1973	59	76.7

Table 5: Best development-sample configurations for Strategy A (plain ARMA) and Strategy B (ARMA-GARCH). Sharpe and cumulative return are net of 10 basis-point transaction costs. The development-sample buy-and-hold Sharpe ratio is 0.2093.

The double-down extension is retained by the parsimony selector because its development-sample Sharpe improvement exceeds the 0.05 tolerance. As the test results below make clear, that retention is itself evidence of development-sample overfitting. We report it honestly rather than redesigning the strategy after seeing the test sample: the grid search spans many threshold-mode-multiplier combinations, so the best development configuration is subject to a selection effect of exactly the kind formalised by White (2000) and Romano and Wolf (2005), and its development Sharpe should be read as an upper bound, not an unbiased estimate.

3.3.3 Moving-average benchmark selection

Table 6 reports the ten moving-average configurations. All produce negative development Sharpe ratios, consistent with the post-2000 decay of technical-rule profits documented by Bajgrowicz and Scaillet (2012). The least-bad configuration, (S, L, B) = (20, 100, 0), is carried forward.

S	L	B	Cum Return	Ann Return	Net Sharpe	Trades
2	10	0.0000	−0.9031	−0.1124	−0.5584	705
2	10	0.0100	−0.9435	−0.1384	−0.8123	1,236
5	20	0.0000	−0.6523	−0.0509	−0.3336	316
5	20	0.0100	−0.7765	−0.0722	−0.4693	597
5	50	0.0000	−0.4414	−0.0281	−0.2523	177
5	50	0.0100	−0.3900	−0.0238	−0.2494	369
10	50	0.0000	−0.2763	−0.0156	−0.2061	143
10	50	0.0100	−0.5075	−0.0341	−0.3062	277
20	100	0.0000	0.0083	0.0004	−0.1487	75
20	100	0.0100	−0.2879	−0.0164	−0.2243	137

Table 6: Development-sample performance of dual moving-average rules on MRK closing prices (80/20 split).

3.3.4 Out-of-sample test performance

Table 7 reports out-of-sample results for the 1,309-day test window (15 October 2020 to 31 December 2025). Buy-and-hold returns a cumulative 37.1% (annualised 8.7%, Sharpe 0.21). Strategy A returns −54.2% net (annualised −8.0%, Sharpe −0.32) on 524 trades; Strategy B returns −9.8% (annualised −0.3%, Sharpe −0.23) on 142 trades; the MA rule returns 10.9% (annualised 2.0%, Sharpe −0.09) on 18 trades. No active strategy beats passive holding.

Strategy	Cum Return	Ann Return	Ann Vol	Sharpe	Trades	Max DD
ARMA net	−0.5424	−0.0797	0.3781	−0.3165	524	0.6311
ARMA-GARCH net	−0.0983	−0.0029	0.1847	−0.2325	142	0.2875
MA(20, 100) net	0.1091	0.0200	0.2303	−0.0870	18	0.5672
Buy-and-Hold (ARMA test)	0.3711	0.0872	0.2296	0.2056	0	0.4474

Strategy	Cum Return	Ann Return	Ann Vol	Sharpe	Trades	Max DD
Buy-and-Hold (MA test)	0.3963	0.0643	0.2303	0.1056	0	0.4474

Table 7: Out-of-sample performance on the test sample (15 October 2020 to 31 December 2025, 1,309 observations). Returns are net of 10 basis-point transaction costs. Sharpe ratios use a 4% annual risk-free rate.

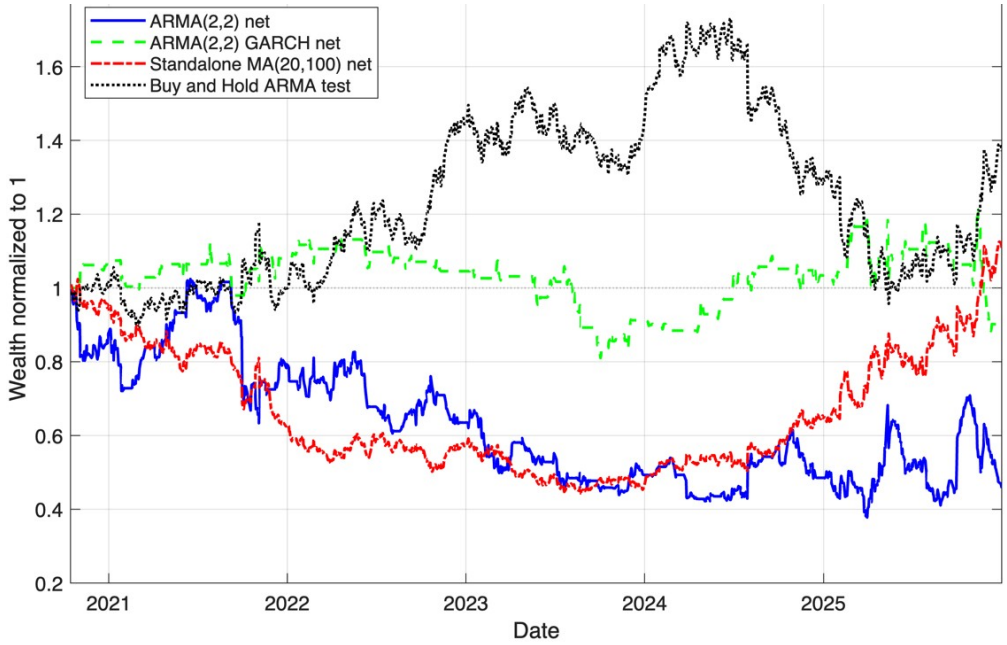

Figure 5: Cumulative wealth, test sample, TC = 10 bps.

Directional accuracy is essentially a coin toss: Strategy A achieves overall and active-signal hit rates of 50.27% and 50.14%, and Strategy B 49.43% and 50.89% — all indistinguishable from 50% (Pesaran and Timmermann, 1992). The decomposition of losses is instructive. Strategy A's gross return is marginally negative at −22.7%, but heavy turnover from the doubled-position rule drives the net figure to −54.2%: the configuration selected on the development sample is precisely the one most punished by costs, the overfitting outcome anticipated in Section 3.3.2. Strategy B's forecast distribution is strongly asymmetric (76.8% positive, skewness −4.56) because the ARMA constant absorbs the equity risk premium; as a result the strategy sits flat 61.4% of the test sample. This is a structural feature of fitting the mean equation to raw rather than excess returns — an obvious refinement for future work would be to demean returns or model excess returns directly, freeing the threshold rule from the risk-premium offset.

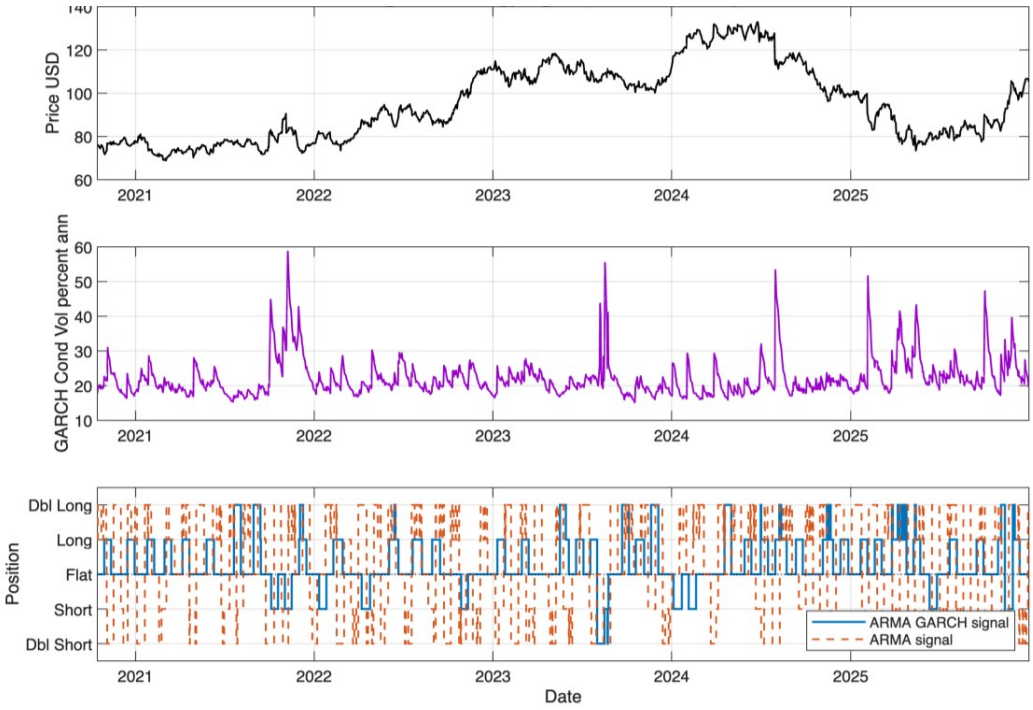

Figure 6: Strategy signals, ARMA test sample.

3.3.5 Pairwise statistical tests

Table 8 reports HAC mean-return differential tests and Ledoit-Wolf Sharpe-ratio tests across all relevant pairs. No comparison yields a p-value below 0.10: although point estimates clearly favour buy-and-hold, the noise in daily return differentials prevents formal rejection of equal performance. The correct reading is therefore two-sided — the active strategies demonstrably fail to add value, and the data cannot even establish that they significantly destroy it. Both readings are consistent with weak-form efficiency.

Comparison	HAC stat	HAC p	LW stat	LW p
ARMA vs ARMA-GARCH	−0.4117	0.6806	−0.1381	0.8902
ARMA vs MA	−0.6632	0.5072	−0.5764	0.5644
ARMA vs Buy-and-Hold	−0.8326	0.4051	−0.8073	0.4195
ARMA-GARCH vs MA	−0.3710	0.7106	−0.4157	0.6776
ARMA-GARCH vs B-and-H	−0.8302	0.4065	−0.8547	0.3927
MA vs Buy-and-Hold	−0.3159	0.7521	−0.3159	0.7521

Table 8: Pairwise tests of equal mean net returns (HAC, Newey-West) and equal Sharpe ratios (Ledoit-Wolf). Two-sided p-values. The null cannot be rejected at any conventional significance level for any pair.

3.3.6 Transaction cost sensitivity

Table 9 reports each strategy's Sharpe ratio across transaction costs from 0 to 50 basis points. At zero cost the ARMA Sharpe is −0.05 and the ARMA-GARCH Sharpe is −0.08. Numerical break-even costs are: ARMA N/A (gross return already negative), ARMA-GARCH ≈ 2.7 bps, MA ≈ 67.5 bps.

TC (bps)	ARMA	ARMA-GARCH	MA	BH (ARMA test)	BH (MA test)
0	−0.0497	−0.0845	−0.0720	0.2056	0.1056

TC (bps)	ARMA	ARMA-GARCH	MA	BH (ARMA test)	BH (MA test)
5	−0.1831	−0.1585	−0.0795	0.2056	0.1056
10	−0.3165	−0.2325	−0.0870	0.2056	0.1056
15	−0.4497	−0.3064	−0.0946	0.2056	0.1056
20	−0.5826	−0.3802	−0.1021	0.2056	0.1056
30	−0.8471	−0.5272	−0.1171	0.2056	0.1056
50	−1.3665	−0.8172	−0.1472	0.2056	0.1056

Table 9: Annualised net Sharpe ratio of each strategy across transaction cost assumptions, in basis points per position-unit change. Buy-and-hold benchmarks are cost-invariant and shown for reference.

The MA rule trades only 18 times and is largely insensitive to costs; its underperformance is driven by directional inaccuracy, not friction. Strategy A, with 524 position turns, loses roughly 0.027 of annualised Sharpe per additional basis point of cost. Critically, zero-cost performance does not lift either ARMA strategy above buy-and-hold: the underperformance reflects a genuine absence of exploitable predictability, not punitive frictions.

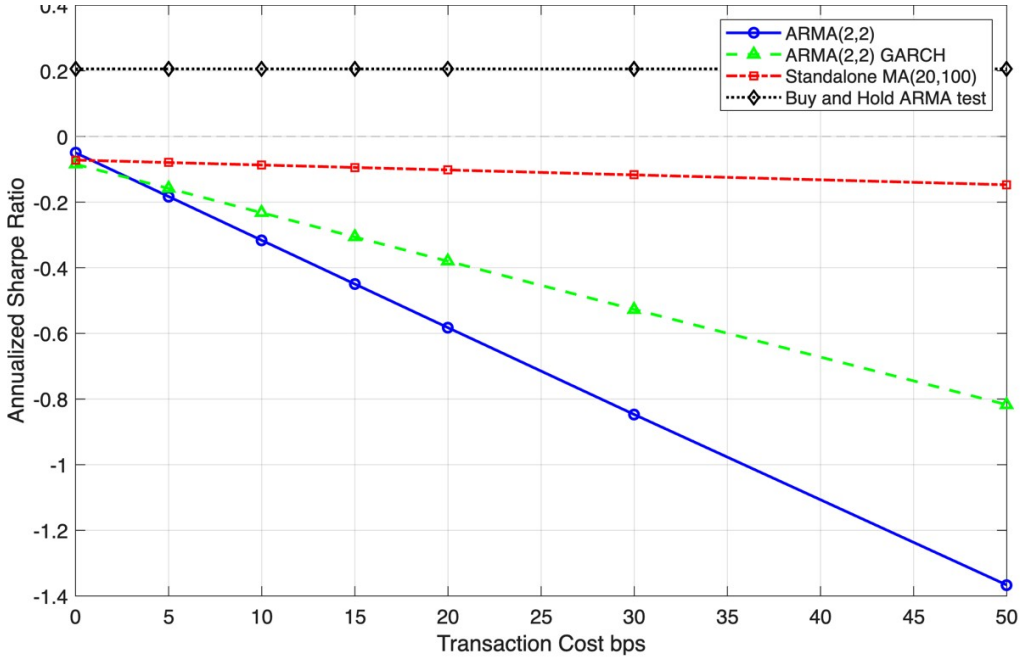

Figure 7: Sharpe ratio vs transaction cost.

3.4 Interpretation

Three consistent facts emerge: hit rates statistically indistinguishable from a coin toss, negative Sharpe ratios even at zero transaction costs, and pairwise tests that cannot separate any strategy from any other. Together with the near-zero autocorrelations of Section 2, this is exactly the configuration the weak-form efficient market hypothesis predicts (Fama, 1970; 1991): whatever linear structure exists in MRK returns at longer lags is too weak, too unstable, or too concentrated in high-cost trading episodes to be converted into risk-adjusted profit. A year-by-year decomposition (Appendix A.2) confirms that no active strategy delivers consistently positive returns in any sub-period, and that the headline losses are not the artefact of a single bad year.

4. Spread Forecasting

4.1 Introduction

This section forecasts the daily price spread of MRK, defined as the difference between the log high and log low price:

$$S_t = log(H_t) - log(L_t) \qquad (8)$$

where H_t and L_t denote the daily high and low prices. The spread captures the intraday range and can therefore serve as a range-based proxy for asset volatility. Since true volatility is unobservable, such proxies are widely used to model and forecast its dynamics; accurate volatility forecasts matter across portfolio allocation, risk management, and derivative pricing. Parkinson (1980) shows that extreme-value (high-low) estimators are considerably more efficient than close-to-close estimators, making the daily spread a credible volatility proxy. Because the spread inherits the key stylised facts of volatility discussed by Corsi (2009) — persistence and clustering, where high-volatility periods follow high-volatility periods — an appropriate model must capture this persistent dependence structure. We model and forecast the one-day-ahead spread using three specifications and compare forecasting performance against a simple AR(1) benchmark.

4.2 Preliminary Analysis

As preliminary checks, we plot the daily spread, its distribution, and its sample ACF.

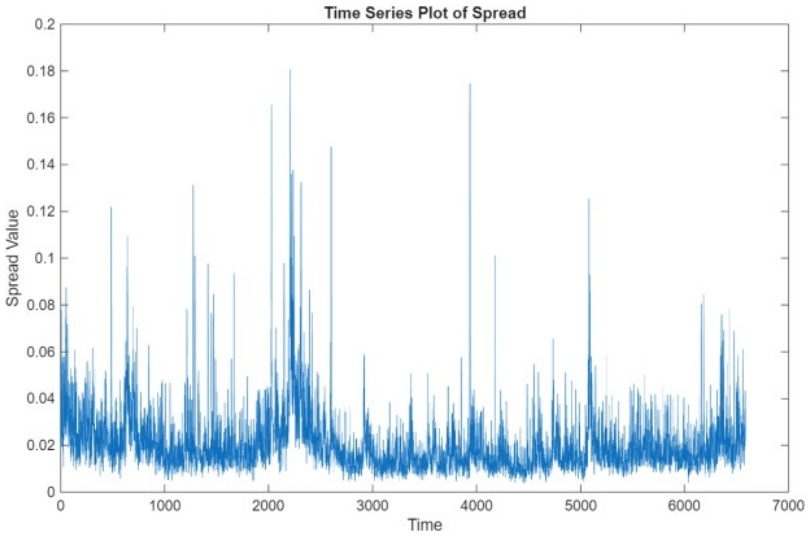

Figure 8(a): Time series plot of MRK spread.

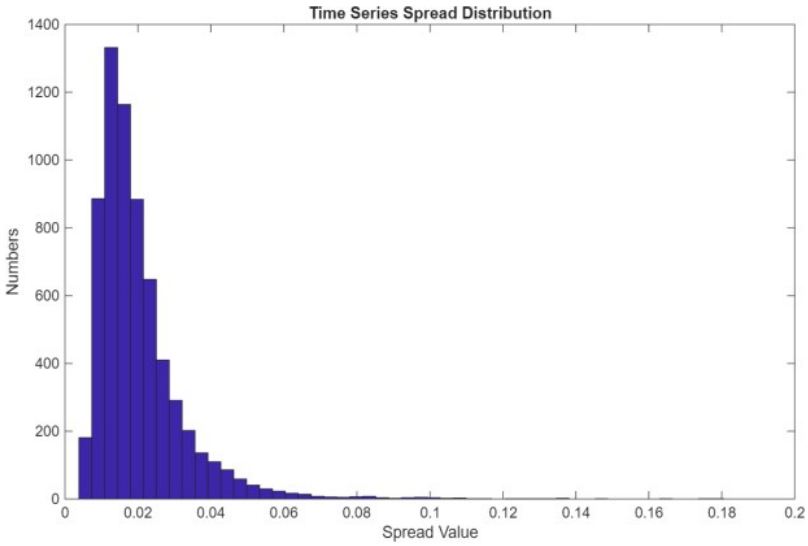

Figure 8(b): Distribution of the MRK spread.

Figure 8(a) shows clear volatility clustering: high- and low-volatility periods persist. The distribution in *Figure 8(b)* is strictly positive and right-skewed, with occasional large spikes likely driven by firm-specific news or market-wide stress.

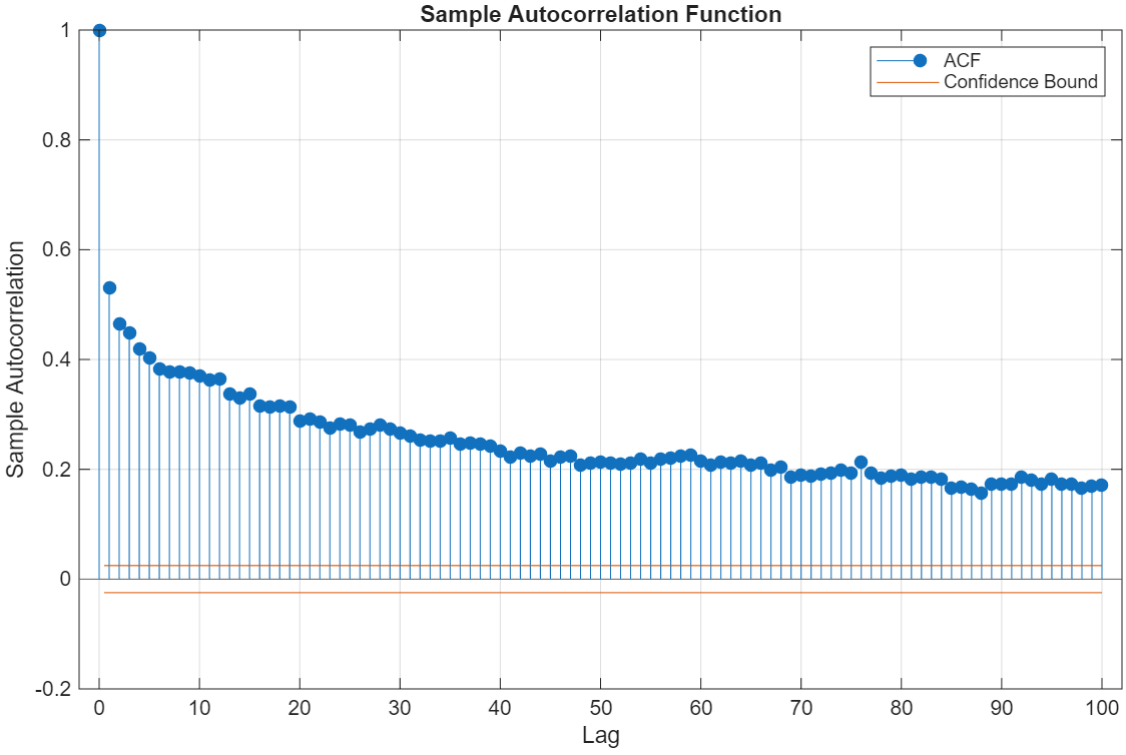

Figure 9: Sample autocorrelation function of the MRK spread.

Figure 9 provides further evidence of persistence: the ACF decays very slowly across 100 lags, indicating long-memory-like behaviour in which past shocks (e.g. market crashes) influence volatility for an extended period. This supports models that capture dependence across multiple horizons rather than a single lag.

4.3 Methodology

Based on the preliminary analysis we consider three models: Log-AR(1), Log-HAR, and Log-ARMA. As the spread is strictly positive and right-skewed, all models are estimated on the log-transformed spread:

$$y_t = log(S_t) \qquad (9)$$

The log transformation reduces the influence of extreme observations and guarantees that back-transformed forecasts remain positive — a necessary property for a volatility-type variable.

4.3.1 Log-AR(1) Model

The Log-AR(1) model serves as the benchmark, providing a simple one-lag forecast:

$$y_t = c + \varphi\, y_{t-1} + \varepsilon_t \qquad (10)$$

AR(1) captures short-run persistence but, by construction, cannot capture dependence across longer horizons — a limitation made visible by the slow-decaying ACF in the preliminary analysis.

4.3.2 Log-HAR Model

To capture persistence across multiple time horizons we employ the Heterogeneous Autoregressive (HAR) model of Corsi (2009), which extends the AR structure with daily, weekly, and monthly components:

$$y_t = \beta_0 + \beta\_d\, y_{t-1} + \beta\_w\, \bar{y}_{t-1:t-5} + \beta\_m\, \bar{y}_{t-1:t-22} + \varepsilon_t \qquad (11)$$

where y_{t-1} is the daily lag, $\bar{y}_{t-1:t-5}$ the average weekly log spread and $\bar{y}_{t-1:t-22}$ the average monthly log spread. HAR is particularly suitable because volatility-type variables display persistent dependence across horizons: Corsi (2009) shows that the model reproduces the long-memory behaviour of volatility parsimoniously by combining time-scale-specific components. The economic rationale rests on the heterogeneous market hypothesis, under which financial markets comprise agents operating at different horizons (Müller et al., 1993); the interaction of these agents generates persistent volatility dynamics of exactly the form HAR is built to capture.

4.3.3 Log-ARMA Model

An ARMA model is also estimated as a flexible, if less theoretically motivated, comparator:

$$y_t = c + \Sigma_{i=1}^{p} \varphi_i\, y_{t-i} + \Sigma_{j=1}^{\varphi} \theta_j\, \varepsilon_{t-j} + \varepsilon_t \qquad (12)$$

The moving-average terms allow shocks to propagate beyond what pure AR dynamics permit, offering a short-memory approximation to the persistence in the data. Orders are selected by minimising the Bayesian Information Criterion (BIC) (Hyndman and Athanasopoulos, 2018). One point of specification deserves emphasis up front: on the full sample the BIC selects ARMA(2,2), whereas on the initial 2,000-observation rolling estimation window it selects ARMA(1,2). The full-sample ARMA(2,2) results are reported in Appendix A.4 for completeness; all rolling-window forecasting below uses ARMA(1,2), the order selected using only in-sample information available at the forecast origin.

4.3.4 Back-Transformation

Because all models are estimated on the log scale, one-day-ahead forecasts are transformed back to the level scale using a log-normal bias correction:

$$\hat{S}_{t+1|t} = exp(\hat{y}_{t+1|t} + \sigma^2_{\varepsilon,t} / 2) \qquad (13)$$

where $\sigma^2_{\varepsilon,t}$ is the residual variance from the corresponding model's in-sample estimation. The correction is required because the exponential of a log-scale forecast does not, in general, equal the conditional mean forecast in levels; naively exponentiating would systematically under-predict the spread. All forecast evaluation is then conducted on the original spread scale.

4.4 Rolling-Window Forecasting Design

Forecasts are produced with a rolling-window design. The dataset contains 6,584 observations; the first 30% (approximately 2,000 days) forms the initial estimation window and the remaining 70% is reserved for out-of-sample one-day-ahead forecasts. At each forecast date the Log-HAR coefficients are re-estimated on the most recent 2,000 observations, a one-day-ahead forecast is produced, and the window rolls forward one day. For the Log-AR(1) and Log-ARMA(1,2) models, coefficients are estimated once on the initial 2,000-observation sample: the conditioning data is updated as the forecast origin rolls forward, but the coefficients and the bias-correction variances are held fixed throughout, due to computational constraints. This asymmetry mildly favours HAR and is discussed as a limitation in Appendix A.5.

Forecast accuracy is evaluated with two loss functions — Mean Squared Prediction Error (MSPE) and Quasi-Likelihood (QLIKE), defined in Appendix A.3. QLIKE is included because it is robust to noise in the volatility proxy and penalises under-prediction more heavily, which is the economically dangerous direction for risk management. Forecast efficiency is tested with the Mincer-Zarnowitz (MZ) regression of the realised value on the forecast:

$$S_t = c + \beta \hat{S}_t + \varepsilon_t \qquad (14)$$

A Wald test evaluates the joint restriction H_0: $c = 0$, $\beta = 1$. Failure to reject indicates unbiased and efficient forecasts; rejection indicates that the model does not fully capture realised spread dynamics. The MZ regressions are run on the level scale, matching the scale on which losses are computed. Finally, Diebold-Mariano (DM) tests (Diebold and Mariano, 1995) assess whether loss differences between model pairs are statistically significant; because all forecasts are one-step-ahead, the loss differential requires no serial-correlation correction beyond lag zero under the null. With a lower loss indicating the better model, a positive and significant DM statistic favours the second model of each pair, and vice versa.

4.5 Empirical Results

4.5.1 Full-Sample Estimation

Before the rolling-window exercise, all three models are estimated on the full sample to examine the dependence structure of the spread and confirm that the specifications are appropriate. Full estimation output is reported in Appendix A.4; in summary, every coefficient in every model is statistically significant, the HAR weekly and monthly components carry substantial explanatory power, and only the AR(1) leaves visible structure in its residual ACF.

4.5.2 Rolling-Window Forecasting Results

Table 10 reports MSPE and QLIKE for the three specifications. The Log-AR(1) benchmark performs worst overall — intuitively so, since it conditions on the previous day's spread alone and ignores the longer-horizon persistence evident in the data. Both Log-HAR and Log-ARMA(1,2) improve substantially: HAR by directly modelling multi-horizon persistence, ARMA by combining autoregressive and moving-average dynamics. Log-HAR attains the lowest MSPE and QLIKE, although its margin over Log-ARMA is minimal.

Model	MSPE (10^{-4})	QLIKE
Log-HAR	1.0021	0.0824
Log-ARMA(1,2)	1.0074	0.0829
Log-AR(1)	1.26	0.1044

Table 10: MSPE and QLIKE results.

The Mincer-Zarnowitz results in *Table 11* support these conclusions. Both Log-HAR and Log-ARMA achieve a markedly higher R^2 than the benchmark, confirming that their forecasts track the realised spread more closely. All three models nonetheless reject the Wald null, so none of the forecasts is fully efficient in the strict MZ sense — the slope estimates above one indicate that all models under-react to movements in the realised spread. Log-HAR has the smallest Wald statistic, followed by Log-ARMA(1,2), with Log-AR(1) far behind: while both richer models are clearly superior to AR(1), HAR comes closest to forecast efficiency.

Model	Intercept (10^{-3})	Forecast	Wald Statistic	p-value	MZ R^2
Log-HAR	−1.4807	1.0936	28.0825	<0.0001	0.4163
Log-ARMA(1,2)	−3.0315	1.1403	53.1916	<0.0001	0.4164
Log-AR(1)	−11.569	1.4771	312.3190	<0.0001	0.3087

Table 11: Mincer-Zarnowitz regression and Wald test results.

Tables 12 and 13 report Diebold-Mariano tests of relative forecast accuracy. Under MSPE loss (*Table 12*), the difference between Log-ARMA(1,2) and Log-HAR is not statistically significant, while both significantly outperform the Log-AR(1) benchmark.

Comparison	DM statistic	p-value	Interpretation
Log-ARMA(1,2) vs Log-HAR	1.0816	0.2794	No significant difference
Log-AR(1) vs Log-HAR	8.7529	<0.0001	Log-HAR significantly outperforms Log-AR(1)
Log-AR(1) vs Log-ARMA(1,2)	9.0289	<0.0001	Log-ARMA(1,2) significantly outperforms Log-AR(1)

Table 12: Diebold-Mariano tests using MSPE loss.

Under QLIKE loss (*Table 13*), the pattern is identical: no significant difference between Log-HAR and Log-ARMA, and decisive rejection against the AR(1) benchmark for both.

Comparison	DM statistic	p-value	Interpretation
Log-ARMA(1,2) vs Log-HAR	1.2107	0.2260	No significant difference
Log-AR(1) vs Log-HAR	14.26	<0.0001	Log-HAR significantly outperforms Log-

Comparison	DM statistic	p-value	Interpretation
			AR(1)
Log-AR(1) vs Log-ARMA(1,2)	15.248	<0.0001	Log-ARMA(1,2) significantly outperforms Log-AR(1)

Table 13: Diebold-Mariano tests using QLIKE loss.

Overall, Log-HAR is the best empirical model, with Log-ARMA(1,2) essentially tied; both significantly outperform the Log-AR(1) benchmark under every criterion considered.

4.6 Discussion

The results align closely with the stylised facts of volatility: the spread is persistent, right-skewed, and clustered, with occasional large spikes. The Log-AR(1) benchmark performs worst on every metric — MSPE, QLIKE, and the MZ diagnostics — reflecting its inability to capture multi-period dependence. Both Log-HAR and Log-ARMA improve materially. HAR's strength is consistent with Corsi (2009): daily, weekly and monthly components parsimoniously reproduce long-memory-like volatility dynamics. That ARMA delivers near-identical accuracy is itself informative — it shows that, at the one-day horizon, a flexible short-memory model can mimic the forecasting content of an explicitly multi-horizon one, so the practical case for HAR here rests on its parsimony and interpretability rather than raw accuracy.

The DM tests confirm that HAR does not significantly outperform ARMA under either loss function, even though both dominate the benchmark decisively. The narrowness of HAR's margin may partly reflect the estimation asymmetry noted in Section 4.4 — HAR was re-estimated daily while ARMA's coefficients were frozen — which, if anything, understates ARMA's achievable performance. Further limitations are discussed in Appendix A.5.

5. Conclusion

This report set out to characterise the time series behaviour of Merck & Co. stock and to test how far that behaviour is predictable. The evidence assembles into a single coherent conclusion built on three findings.

First, MRK daily log returns are stationary but decisively non-Gaussian — leptokurtic with excess kurtosis near 28, mildly negatively skewed — and are serially uncorrelated at short lags, with only weak dependence emerging beyond lag 14. Second, that weak dependence has no economic value: rolling-window ARMA and ARMA-GARCH trading rules, selected under a strict development/test protocol, fail to beat buy-and-hold out of sample, with coin-toss hit rates and negative Sharpe ratios that persist even at zero transaction costs. Third, in sharp contrast, the conditional second moment is highly forecastable: Log-HAR and Log-ARMA models cut the one-day-ahead spread forecasting loss substantially relative to an AR(1) benchmark, with Diebold-Mariano p-values below 0.0001 under both MSPE and QLIKE loss.

Together these results reproduce, in a single security, the central asymmetry of modern financial econometrics: returns are unpredictable in mean but predictable in variance (Fama, 1970; Corsi, 2009). The practical implication is that modelling effort on daily equity data is better spent on risk — volatility forecasting for position sizing, risk management, and derivative pricing — than on directional trading

signals. Natural extensions include modelling excess rather than raw returns in the trading exercise, applying formal multiple-testing corrections to the strategy search, fully re-estimating all spread models in each rolling window, and augmenting the spread models with exogenous predictors such as volume and market-wide volatility.

Appendix

A.1 Power Spectral Density (supplement to Section 2.4)

The periodogram, first introduced by Schuster (1898), estimates the power spectral density (PSD) as the squared magnitude of the discrete-time Fourier transform of the series, normalised by frequency resolution. It shows which frequencies contain the largest signal: under white noise the spectrum should be approximately flat. Fisher (1929) used this device to test for white noise via the g-statistic, and Welch (1967) refined the estimator by averaging modified periodograms over overlapping segments. *Figure A1* reports both estimates for MRK returns.

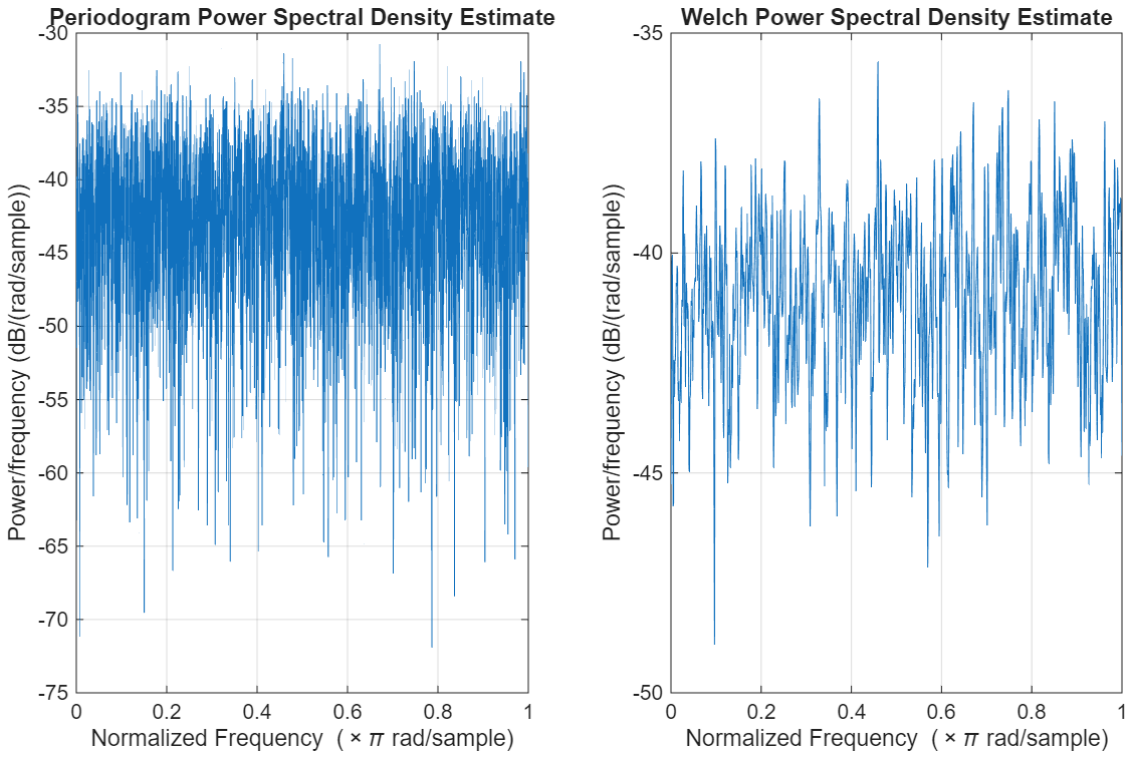

Figure A1: Periodogram (left) and Welch (right) PSD estimates of MRK daily log returns.

Both the periodogram and the Welch estimate fluctuate broadly around a constant level without obvious trends or peaks, consistent with a (near-)white-noise spectrum and corroborating the time-domain evidence of Section 2.4.

A.2 Year-by-Year Decomposition (supplement to Section 3.3)

Table A1 reports annualised net returns by calendar year. Strategy A performed worst in 2020 (−55.8%, driven by the post-vaccine rally stopping out positions in the late-October to year-end window), while Strategy B benefited from the same window (+34.2%). In 2024-2025 the MA rule recovers strongly while both ARMA strategies oscillate near zero, suggesting the longer-horizon trend signal captured something the ARMA mean equation missed. No active strategy delivers consistently positive year-on-year returns, and buy-and-hold is the most stable.

Year	ARMA net	ARMA-GARCH net	MA net	BH (ARMA)	BH (MA)
2020	−55.75	34.18	−28.66	9.24	16.20
2021	−2.45	1.85	−40.40	1.02	−1.76

Year	ARMA net	ARMA-GARCH net	MA net	BH (ARMA)	BH (MA)
2022	−18.27	−2.72	−9.48	39.13	37.14
2023	−21.03	−12.30	−23.27	−0.01	−1.77
2024	3.61	13.41	35.42	−6.89	−9.16
2025	8.53	−9.27	54.32	10.30	5.69

Table A1: Annualised net returns by calendar year on the out-of-sample test period (percent per annum). Years are computed from realised daily returns on the dates each strategy is active and may differ in length where strategies use slightly different sample splits.

A.3 Loss Functions (supplement to Section 4.4)

The first loss function is the Mean Squared Prediction Error:

$$MSPE = (1/T)\ \Sigma_{t=1}^{T}\ (S_t - \hat{S}_t)^2 \qquad \text{(A1)}$$

where T is the number of out-of-sample forecasts; lower MSPE indicates better accuracy. The second is the Quasi-Likelihood loss:

$$QLIKE = (1/T)\ \Sigma_{t=1}^{T}\ [\ S_t/\hat{S}_t - ln(S_t/\hat{S}_t) - 1\ ] \qquad \text{(A2)}$$

QLIKE is most suitable when true volatility is unobserved and must be evaluated against a noisy proxy; unlike MSPE it is asymmetric, penalising under-prediction of volatility more heavily. Both are standard robust loss functions for evaluating volatility forecasts.

A.4 Full-Sample Estimation (supplement to Section 4.5.1)

The full-sample estimates confirm that Log-AR(1) captures some short-run persistence, with a positive and significant lag coefficient; however, its residual ACF retains clear serial correlation, indicating that a single lag does not exhaust the dependence structure and supporting the richer specifications. The Log-ARMA(2,2), selected by BIC on the full sample, has all AR and MA coefficients statistically significant and removes most of the residual autocorrelation left by AR(1). The Log-HAR model provides the strongest framework: the daily, weekly and monthly components are all statistically significant, with the weekly and monthly terms carrying the more substantial explanatory power — recent medium- and longer-horizon information is genuinely useful for forecasting the next day's spread. The model achieves an R^2 of approximately 0.3901, so 39.01% of the variation in the daily spread is explained by the three volatility components, consistent with Corsi's (2009) argument that HAR captures persistent volatility dynamics through a simple multi-horizon autoregressive structure.

A.4.1 Log-AR(1) Model

	Value	StandardError	TStatistic	PValue
Constant	-1.9133	0.039526	-48.405	0
AR{1}	0.52585	0.0098149	53.577	0
Variance	0.18559	0.0029612	62.674	0

Table A2: Full-sample Log-AR(1) estimation.

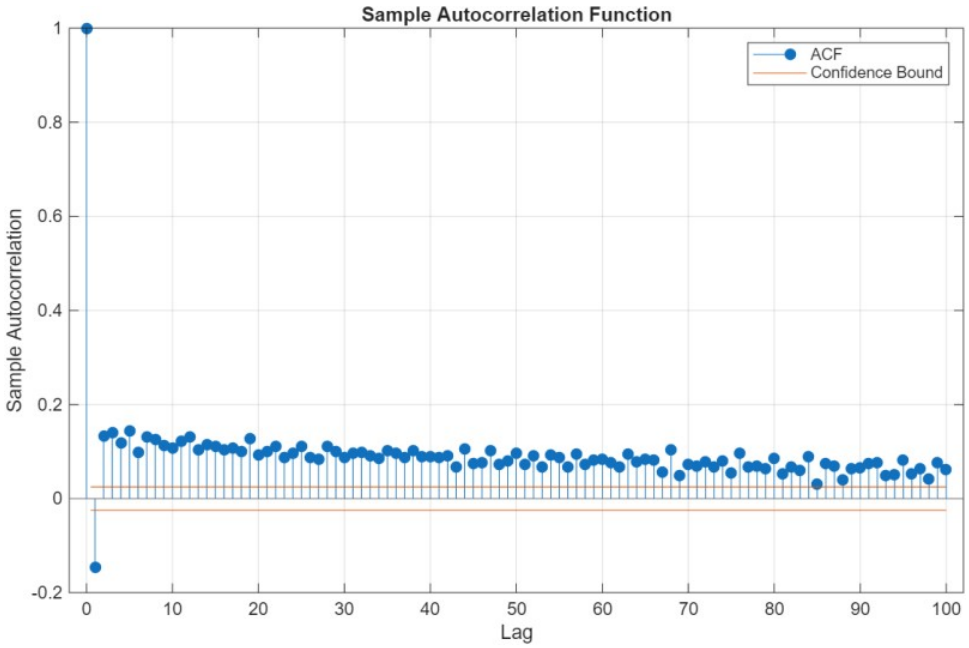

Figure A2: Sample autocorrelation function of Log-AR(1) residuals.

The residual autocorrelation plot shows that past values remain correlated with future ones — long-memory-like behaviour that a single autoregressive lag cannot absorb.

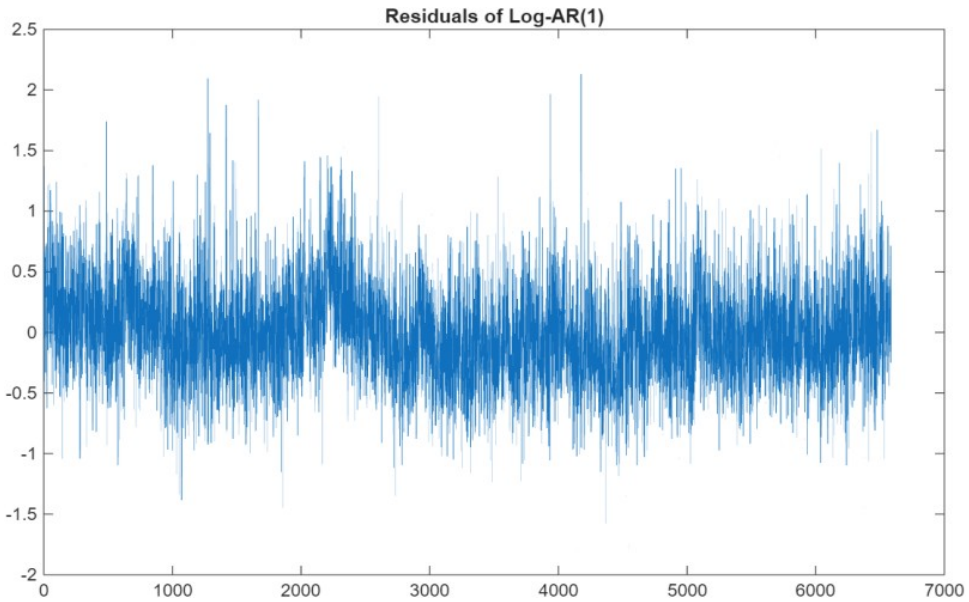

Figure A3: Residuals of the Log-AR(1) model.

A.4.2 Log-ARMA Model

On the full sample, information criteria select Log-ARMA(2,2) as the minimum-BIC specification:

	Value	StandardError	TStatistic	PValue
Constant	-0.007694	0.0023488	-3.2758	0.0010537
AR{1}	1.7192	0.027489	62.54	0
AR{2}	-0.72107	0.027197	-26.513	6.7853e-155
MA{1}	-1.4788	0.032117	-46.045	0
MA{2}	0.50444	0.028923	17.441	4.0285e-68
Variance	0.15474	0.0023349	66.273	0

Table A3: Full-sample Log-ARMA(2,2) estimation.

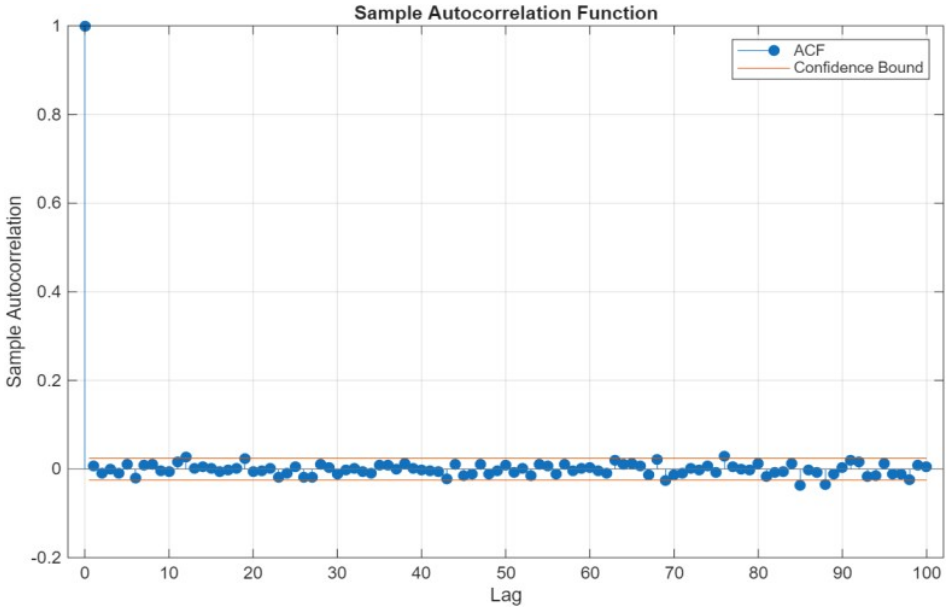

Figure A4: Sample autocorrelation function of Log-ARMA(2,2) residuals.

A.4.3 Log-HAR Model

	Estimate	SE	tStat	pValue
(Intercept)	-0.43407	0.059067	-7.3488	2.2431e-13
y1_d_lag1	0.17979	0.014556	12.351	1.1613e-34
y1_w	0.30616	0.027543	11.116	1.8855e-28
y1_m	0.40663	0.027109	15	4.9579e-50

Number of observations: 6562, Error degrees of freedom: 6558
Root Mean Squared Error: 0.395
R-squared: 0.39, Adjusted R-Squared: 0.39
F-statistic vs. constant model: 1.4e+03, p-value = 0

Table A4: Full-sample HAR model estimation.

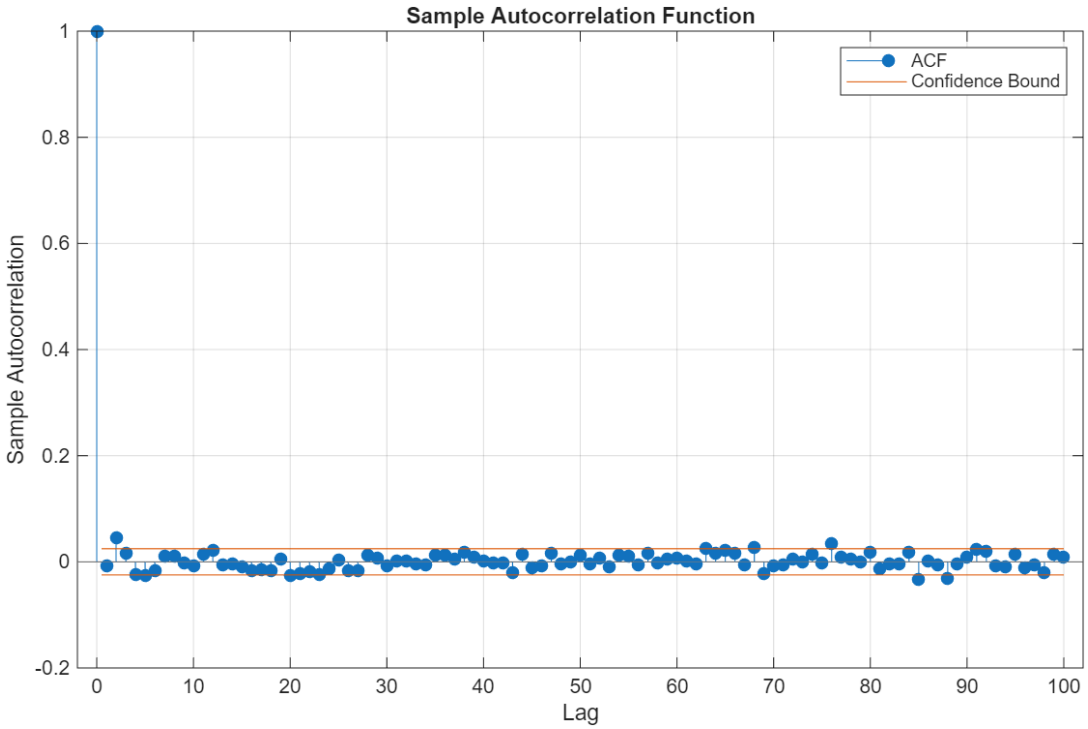

Figure A5: Sample autocorrelation function of Log-HAR standardised residuals.

A.5 Limitations

Three limitations qualify the spread-forecasting results. First, all models condition only on the past spread; forecasting could plausibly be improved by additional predictors such as trading volume, market-wide volatility, and returns. Even so, the rolling-window evidence establishes that the daily spread behaves as a persistent volatility proxy and that Log-HAR and Log-ARMA are clearly superior to the AR(1) benchmark for one-day-ahead forecasting.

Second, the estimation design is asymmetric. The Log-HAR model is re-estimated in every rolling window, whereas the Log-AR(1) and Log-ARMA(1,2) coefficients — and the residual variances used in the log-normal bias correction — were estimated once on the initial 2,000-observation sample and held fixed thereafter, due to computational and time constraints. The AR(1) and ARMA forecasts are therefore less able to adapt to changes in spread dynamics over time, and a fully re-estimated implementation could improve their accuracy. Notably, even without re-estimation, Log-ARMA(1,2) performs nearly identically to Log-HAR and substantially outperforms Log-AR(1), which strengthens rather than weakens the central conclusion that richer dynamic models provide better one-day-ahead forecasts than the simple benchmark.

Third, the fixed bias-correction variance interacts with the second point: if the true residual variance drifts over the out-of-sample period, the level-scale forecasts of the frozen models inherit a slowly varying bias. This is one candidate explanation for the MZ slope coefficients exceeding one for all models, and re-estimating the correction alongside the coefficients is the natural remedy.

References

Bajgrowicz, P. and Scaillet, O. (2012) ‘Technical trading revisited: false discoveries, persistence tests, and transaction costs’, Journal of Financial Economics, 106(3), pp. 473–491.

Brock, W., Lakonishok, J. and LeBaron, B. (1992) ‘Simple technical trading rules and the stochastic properties of stock returns’, Journal of Finance, 47(5), pp. 1731–1764.

Burnham, K.P. and Anderson, D.R. (2002) Model selection and multimodel inference: a practical information-theoretic approach. 2nd edn. New York: Springer.

Corsi, F. (2009) ‘A simple approximate long-memory model of realized volatility’, Journal of Financial Econometrics, 7(2), pp. 174–196.

Diebold, F.X. and Mariano, R.S. (1995) ‘Comparing predictive accuracy’, Journal of Business and Economic Statistics, 13(3), pp. 253–263.

Engle, R.F. (1982) ‘Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation’, Econometrica, 50(4), pp. 987–1007.

Fama, E.F. (1965) ‘The behavior of stock-market prices’, Journal of Business, 38(1), pp. 34–105.

Fama, E.F. (1970) ‘Efficient capital markets: a review of theory and empirical work’, Journal of Finance, 25(2), pp. 383–417.

Fama, E.F. (1991) ‘Efficient capital markets: II’, Journal of Finance, 46(5), pp. 1575–1617.

Fisher, R.A. (1929) ‘Tests of significance in harmonic analysis’, Proceedings of the Royal Society of London, Series A, 125(796), pp. 54–59.

Hansen, P.R. and Lunde, A. (2005) ‘A forecast comparison of volatility models: does anything beat a GARCH(1,1)?’, Journal of Applied Econometrics, 20(7), pp. 873–889.

Hyndman, R.J. and Athanasopoulos, G. (2018) Forecasting: principles and practice. 2nd edn. Melbourne: OTexts. Available at: https://otexts.com/fpp2 (Accessed: 13 May 2026).

Jarque, C.M. and Bera, A.K. (1987) ‘A test for normality of observations and regression residuals’, International Statistical Review, 55(2), pp. 163–172.

Ledoit, O. and Wolf, M. (2008) ‘Robust performance hypothesis testing with the Sharpe ratio’, Journal of Empirical Finance, 15(5), pp. 850–859.

Ljung, G.M. and Box, G.E.P. (1978) ‘On a measure of lack of fit in time series models’, Biometrika, 65(2), pp. 297–303.

Lo, A.W. and MacKinlay, A.C. (1988) ‘Stock market prices do not follow random walks: evidence from a simple specification test’, Review of Financial Studies, 1(1), pp. 41–66.

Müller, U., Dacorogna, M., Davé, R., Pictet, O., Olsen, R. and Ward, J. (1993) Fractals and intrinsic time — a challenge to econometricians. Olsen & Associates Discussion Paper, pp. 22–26.

Newey, W.K. and West, K.D. (1987) ‘A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix’, Econometrica, 55(3), pp. 703–708.

Parkinson, M. (1980) ‘The extreme value method for estimating the variance of the rate of return’, Journal of Business, 53(1), pp. 61–65.

Pesaran, M.H. and Timmermann, A. (1992) ‘A simple nonparametric test of predictive performance’, Journal of Business and Economic Statistics, 10(4), pp. 461–465.

Pesaran, M.H. and Timmermann, A. (1995) ‘Predictability of stock returns: robustness and economic significance’, Journal of Finance, 50(4), pp. 1201–1228.

Romano, J.P. and Wolf, M. (2005) ‘Stepwise multiple testing as formalized data snooping’, Econometrica, 73(4), pp. 1237–1282.

Schuster, A. (1898) ‘On the investigation of hidden periodicities with application to a supposed 26 day period of meteorological phenomena’, Terrestrial Magnetism, 3(1), pp. 13–41.

Taylor, S.J. (2007) Asset price dynamics, volatility, and prediction. Princeton: Princeton University Press.

Welch, P.D. (1967) ‘The use of fast Fourier transform for the estimation of power spectra: a method based on time averaging over short, modified periodograms’, IEEE Transactions on Audio and Electroacoustics, 15(2), pp. 70–73.

White, H. (2000) ‘A reality check for data snooping’, Econometrica, 68(5), pp. 1097–1126.

Code

Task 1 Code

```
%% Stationarity
clearvars; clc; close all;
filename = 'merck_26 years.xlsx';
opts = detectImportOptions(filename,'NumHeaderLines',6);
data = readtable(filename,opts);
data = data(:,{'Date','PX_LAST'});
data = rmmissing(data);


data = sortrows(data,'Date');
logP = log(data.PX_LAST);
ret

= 100*diff(logP);

%% Time series plots
subplot(1,2,1)
plot(logP)
title('MRK Log Prices')
subplot(1,2,2)
plot(ret)
title('MRK Daily Log Returns')
figure
%% Correlograms
subplot(1,2,1)
autocorr(logP,'NumLags',50)
title('ACF of Log Prices')
subplot(1,2,2)
autocorr(ret,'NumLags',50)
title('ACF of Log Returns')
%% ADF tests with lag selection by BIC
[hP,pValueP,statP,cValueP,regP] = adftest(logP,'model','TS','lags',0:20);
[hR,pValueR,statR,cValueR,regR] = adftest(ret,'model','ARD','lags',0:20);
% Extract BIC values
bicP = [regP.BIC];
bicR = [regR.BIC];
% Remove invalid entries if any
bicP(~isfinite(bicP)) = Inf;
bicR(~isfinite(bicR)) = Inf;
% Choose optimal lag by BIC
[~,opt_lagP] = min(bicP);
[~,opt_lagR] = min(bicR);


%% Collect results at optimal lag
opt_lag_prices

= opt_lagP - 1;

opt_lag_returns = opt_lagR - 1;
stat_prices = statP(opt_lagP);
stat_returns = statR(opt_lagR);
p_prices = pValueP(opt_lagP);
p_returns = pValueR(opt_lagR);
h_prices = hP(opt_lagP);
h_returns = hR(opt_lagR);
% Critical values: some MATLAB versions return 1x3, others Nx3
if size(cValueP,1) == 1
crit_prices = cValueP;
else
crit_prices = cValueP(opt_lagP,:);
end
if size(cValueR,1) == 1
crit_returns = cValueR;
else
crit_returns = cValueR(opt_lagR,:);
end
%% Display results
fprintf('\nADF TEST RESULTS\n');
fprintf('------------------------------------------\n');
fprintf('\nMRK LOG PRICES\n');
fprintf('Optimal Lag (BIC): %d\n',opt_lag_prices);
fprintf('Test Statistic: %.4f\n',stat_prices);
fprintf('p-value: %.4f\n',p_prices);
fprintf('Critical Values: 1%%=%.4f

5%%=%.4f

10%%=%.4f\n', ...

crit_prices(1),crit_prices(2),crit_prices(3));
fprintf('Decision (1 = Reject unit root): %d\n',h_prices);
fprintf('\nMRK Log RETURNS\n');
fprintf('Optimal Lag (BIC): %d\n',opt_lag_returns);
fprintf('Test Statistic: %.4f\n',stat_returns);


fprintf('p-value: %.4f\n',p_returns);
fprintf('Critical Values: 1%%=%.4f

5%%=%.4f

10%%=%.4f\n', ...

crit_returns(1),crit_returns(2),crit_returns(3));
```

```
fprintf('Decision (1 = Reject unit root): %d\n',h_returns);
fprintf('-------------------------------------------\n');

clc
%readtable into Matlab
timeseries = readtable('C:\Users\ytshe\OneDrive\desktop\matlab
script\MRK.csv')
y1=timeseries.logReturn
y2=timeseries.Close
subplot(1,2,1)
periodogram(y1)
subplot(1,2,2)
pwelch(y1)
%Basic plot and Description
summary(y1)
subplot(2,2,1)
plot(y1)
xlabel('Daily log return');
ylabel('Numbers');
title('Return distribution')
subplot(2,2,2)
hist(y1,50)
xlabel('Daily log return');
ylabel('Numbers');
title('Return distribution');
subplot(2,2,3)
qqplot(y1)
subplot(2,2,4)
autocorr(y1,'NumLags',100)
subplot(1,2,1)
autocorr(y1,'NumLags',100)
subplot(1,2,2)
parcorr(y1,'NumLags',100)


%Lbq test white noise
[h,pvalue,Q,CV] = lbqtest(y1,'Lags',1:20);
disp(table((1:20)',h',pvalue',Q',CV','VariableNames',{'lags','Reject?','pval','LBstat','5%CV'}))
%Augmented Dickey Fuller test stationary
[h,pvalue, stat, cvalue,reg]=adftest(y1,'Lags',0:20,'Model','TS')
BICvec=[reg.BIC];
[~,pstar]=min(BICvec);
test_stat=stat(pstar)
hstar = h(pstar);
pvalstar = pvalue(pstar);
output=array2table([pstar;test_stat;pvalstar;hstar],'VariableNames',{'Daily
log Return'},'Rownames',{'Opt_lag','Test_stat','p_val','decision'});
disp(output)

X = readmatrix('merck_26 years.csv');
X(:,1) = [];
disp(X)
P = X(:,1);
P = P(~isnan(P));
r = diff(log(P));
r = r(~isnan(r));
figure
histogram(r, 'Normalization','pdf','FaceColor',[0.2,0.6,0.8])
hold on
mu = mean(r);
sigma = std(r);
x = linspace(min(r), max(r), 200);
plot(x, normpdf(x,mu,sigma), 'k-', 'LineWidth', 1.5)
hold off
xlabel('Daily Log Return')
ylabel('Density')
title('Histogram of Daily Log Returns - Merck')
set(groot, 'defaultFigureColor','w')
```

Task 2 Code

```
clc
clear
close all
warning('off','all')
priceTable = readtable('merck.csv');
priceTable.Properties.VariableNames = {'Date','Close'};
priceTable.Date = datetime(priceTable.Date, 'InputFormat', 'dd/MM/yyyy');
priceTable = sortrows(priceTable, 'Date', 'ascend');
price
= priceTable.Close;
date
= priceTable.Date;
logRet = diff(log(price));
simpRet = exp(logRet) - 1;
retDate = date(2:end);
N
= length(logRet);
%% ============================================================
% 2. SAMPLE SPLIT (60 / 20 / 20) — used by ARMA / ARMA-GARCH
%
The MA rule (Section 12) uses Script 1's separate 80/20 split.
% ============================================================
T_train = floor(0.60 * N);
T_dev
= floor(0.20 * N);
T_test = N - T_train - T_dev;
idx.trainStart = 1;
idx.trainEnd
= T_train;
idx.devStart
```

```
= T_train + 1;
idx.devEnd
= T_train + T_dev;
idx.testStart = idx.devEnd + 1;
idx.testEnd
= N;
fprintf('Sample split — Train: %d Dev: %d Test: %d (Total: %d)\n', ...
T_train, T_dev, T_test, N);
fprintf(' Train : %s to %s\n', string(retDate(idx.trainStart)),
string(retDate(idx.trainEnd)));
fprintf(' Dev
: %s to %s\n', string(retDate(idx.devStart)),
string(retDate(idx.devEnd)));
fprintf(' Test : %s to %s\n', string(retDate(idx.testStart)),
string(retDate(idx.testEnd)));
% --- ARMA order search bounds --cfg.maxP = 2;
cfg.maxQ = 2;
% --- Transaction costs and risk-free rate --cfg.tc
= 0.001;
cfg.rfAnnual = 0.04;
cfg.rfDaily = cfg.rfAnnual / 252;
% --- Rolling window settings --cfg.refitEvery = 20;
cfg.winLen
= T_train;
cfg.minHold
= 10;
% --- Threshold grids --cfg.rawGrid = [0.0001, 0.0002, 0.0003, 0.0004, 0.0005, 0.0006, ...
0.00075, 0.001, 0.0015, 0.002, 0.003, 0.005];
cfg.volGrid = [0.005, 0.01, 0.015, 0.02, 0.03, 0.04, 0.05, ...


0.075, 0.10, 0.15, 0.20];
cfg.pctGrid
= [0.50, 0.55, 0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90,
0.95];
cfg.pctLookback = 60;
% --- Double-down settings --cfg.useDoubleDown = true;
cfg.ddMultipliers = [2, 3];
% --- Moving-average benchmark (Taylor 2007 three-state rule) --%
Grid follows Script 1: S in {1, 5, 10}, L in {50, 100}, B in {0, 0.01}.
cfg.S_list = [1, 5, 10];
cfg.L_list = [50, 100];
cfg.B_list = [0, 0.01];
% --- Dev-sample display --cfg.showTopN = 10;
% --- Trivial-signal filters --cfg.minTradesDev
= 25;
cfg.requireBothSides = true;
cfg.minLongPct
= 1.0;
cfg.minShortPct
= 1.0;
cfg.maxTradePct
= 40;
% --- IC tolerance windows --cfg.aicDelta = 2;
cfg.bicDelta = 2;
cfg.hqicDelta = 2;
% --- Parsimony tolerance --cfg.parsimonyTol = 0.05;
% --- TC sensitivity grid --cfg.tcGrid = [0, 0.0005, 0.001, 0.0015, 0.002, 0.003, 0.005];
% --- GARCH settings (Strategy B only) --cfg.sigmaTarget
= NaN;
cfg.garchScaleModes = ["raw", "volscaled"];
cfg.stratBModes
= ["raw", "volscaled"];
if cfg.winLen < 50
error('winLen=%d is too short for reliable estimation.', cfg.winLen);
end
if cfg.minHold < 1
error('minHold must be >= 1.');
end
if cfg.useDoubleDown && any(cfg.ddMultipliers <= 1)
error('ddMultipliers must all be > 1 (thr2 must exceed thr1).');
end
fprintf('\n========== SECTION 5: IC PRE-SCREENING (ARMA ORDER SELECTION)
==========\n');
trainY = logRet(idx.trainStart:idx.trainEnd);
icTable = buildICTable(trainY, cfg.maxP, cfg.maxQ);
disp(icTable(:, {'p','q','AIC','BIC','HQIC','nParams'}))
bestAIC
bestBIC


= min(icTable.AIC);
= min(icTable.BIC);

bestHQIC = min(icTable.HQIC);
icTable.PassAIC = icTable.AIC <= bestAIC + cfg.aicDelta;
icTable.PassBIC = icTable.BIC <= bestBIC + cfg.bicDelta;
icTable.PassHQIC = icTable.HQIC <= bestHQIC + cfg.hqicDelta;
icTable.Votes
= icTable.PassAIC + icTable.PassBIC + icTable.PassHQIC;
fprintf('Best AIC: %.2f
Best BIC: %.2f
Best HQIC: %.2f\n', ...
bestAIC, bestBIC, bestHQIC);
fprintf('\n--- Models passing at least one criterion ---\n');
disp(icTable(icTable.Votes >= 1, :))
fprintf('--- Models passing at least two criteria ---\n');
disp(icTable(icTable.Votes >= 2, :))
allowedModels = table2array(icTable(icTable.Votes >= 1, {'p','q'}));
fprintf('Models retained: %d / %d\n', size(allowedModels,1),
height(icTable));
if isempty(allowedModels)
error('No ARMA models passed the IC pre-screening stage.')
```

```
end
%% ========================================================
% 6. GARCH DIAGNOSTICS ON TRAINING SAMPLE
% ========================================================
fprintf('\n========== SECTION 6: GARCH DIAGNOSTICS (JUSTIFICATION FOR
STRATEGY B) ==========\n');
[~, diagIdx] = min(icTable.AIC);
diagP = icTable.p(diagIdx);
diagQ = icTable.q(diagIdx);
try
mdlDiag
= arima(diagP, 0, diagQ);
mdlDiag.Variance = garch(1, 1);
estDiag
= estimate(mdlDiag, trainY, 'Display','off');
garchParms = estDiag.Variance;
omega = garchParms.Constant;
alpha = garchParms.ARCH{1};
beta = garchParms.GARCH{1};
fprintf('ARMA(%d,%d)-GARCH(1,1) on training sample:\n', diagP, diagQ);
fprintf(' omega
= %.6f\n', omega);
fprintf(' alpha (ARCH) = %.4f\n', alpha);
fprintf(' beta (GARCH) = %.4f\n', beta);
fprintf(' alpha + beta = %.4f (persistence; <1 required)\n', alpha +
beta);
if alpha + beta >= 1
warning('GARCH persistence >= 1 — results may be unreliable.');
end
[trainResid, trainCondVar, ~] = infer(estDiag, trainY);
sqResid = trainResid.^2;
[~, lbP, lbStat] = lbqtest(sqResid, 'Lags', 10);
fprintf(' Ljung-Box Q(10) on squared residuals: stat=%.2f, p=%.4f\n',
lbStat, lbP);
if lbP < 0.05
fprintf(' >> ARCH effects confirmed (p<0.05): GARCH specification
justified for Strategy B.\n');
else
fprintf(' >> No significant ARCH effects (p>=0.05).\n');
end
if isnan(cfg.sigmaTarget)
cfg.sigmaTarget = mean(sqrt(max(trainCondVar, 1e-10)));


fprintf(' sigmaTarget (Strategy B vol-scaling): %.6f\n',
cfg.sigmaTarget);
end
catch ME
fprintf('GARCH diagnostic failed: %s\n', ME.message);
if isnan(cfg.sigmaTarget)
cfg.sigmaTarget = std(trainY);
fprintf(' sigmaTarget resolved from rolling std: %.6f\n',
cfg.sigmaTarget);
end
end
fprintf('\n========== SECTION 7: BUY-AND-HOLD BENCHMARK (DEV SAMPLE)
==========\n');
devBhRet = simpRet(idx.devStart:idx.devEnd);
sBhDev
= summariseReturns(devBhRet, zeros(T_dev,1), cfg.rfDaily);
fprintf('Cum Return : %.4f\n', sBhDev.CumReturn);
fprintf('Ann Return : %.4f\n', sBhDev.AnnReturn);
fprintf('Ann Vol
: %.4f\n', sBhDev.AnnVol);
fprintf('Sharpe
: %.4f\n', sBhDev.Sharpe);
fprintf('\n========== SECTION 8: DEV EVALUATION — STRATEGY A (PLAIN ARMA)
==========\n');
cfgA
= cfg;
cfgA.useGarch
= false;
cfgA.garchScale = false;
cfgA.stratBModes = ["raw","volscaled","percentile"];
[devTableA, cacheA] = evaluateArmaConfigsOnDev( ...
logRet, simpRet, allowedModels, idx, cfgA);
devTableA = sortrows(devTableA, 'SharpeNet', 'descend');
validRowsA = ~devTableA.IsTrivial & isfinite(devTableA.SharpeNet);
fprintf('Total configs: %d
Non-trivial: %d
Trivial: %d\n', ...
height(devTableA), sum(validRowsA), sum(~validRowsA));
if ~any(validRowsA)
error('All Strategy A (plain ARMA) configurations were trivial.')
end
fprintf('\nTop %d configurations — Strategy A (plain ARMA):\n',
cfg.showTopN);
topA = devTableA(validRowsA,:);
disp(topA(1:min(cfg.showTopN, height(topA)), :))
fprintf('\n--- Best per threshold mode (Strategy A) ---\n');
for m = ["raw","volscaled","percentile"]
rows = validRowsA & devTableA.Mode == m & devTableA.DDMult == 1;
if any(rows)
tmp = sortrows(devTableA(rows,:), 'SharpeNet', 'descend');
fprintf('%-12s ARMA(%d,%d) thr=%.5f Sharpe=%.4f
Trades=%.0f\n', ...
m, tmp.p(1), tmp.q(1), tmp.Threshold(1), tmp.SharpeNet(1),
tmp.NTrades(1));
else
fprintf('%-12s no valid configuration\n', m);
end
end
```

```
if cfg.useDoubleDown
fprintf('\n--- Best double-down configs (Strategy A) ---\n');
for mult = cfg.ddMultipliers
rows = validRowsA & devTableA.DDMult == mult;
if any(rows)
tmp = sortrows(devTableA(rows,:), 'SharpeNet', 'descend');
fprintf('DDMult=%.0fx ARMA(%d,%d) mode=%-10s thr1=%.5f
Sharpe=%.4f Trades=%.0f\n', ...
mult, tmp.p(1), tmp.q(1), tmp.Mode(1), tmp.Threshold(1),
tmp.SharpeNet(1), tmp.NTrades(1));
end
end
end
fprintf('\n========== SECTION 9: DEV EVALUATION — STRATEGY B (ARMA-GARCH)
==========\n');
cfgB
= cfg;
cfgB.useGarch
= true;
cfgB.garchScale = true;
[devTableB, cacheB] = evaluateArmaConfigsOnDev( ...
logRet, simpRet, allowedModels, idx, cfgB);
devTableB = sortrows(devTableB, 'SharpeNet', 'descend');
validRowsB = ~devTableB.IsTrivial & isfinite(devTableB.SharpeNet);
fprintf('Total configs: %d
Non-trivial: %d
Trivial: %d\n', ...
height(devTableB), sum(validRowsB), sum(~validRowsB));
if ~any(validRowsB)
error('All Strategy B (ARMA-GARCH) configurations were trivial.')
end
fprintf('\nTop %d configurations — Strategy B (ARMA-GARCH):\n',
cfg.showTopN);
topB = devTableB(validRowsB,:);
disp(topB(1:min(cfg.showTopN, height(topB)), :))
fprintf('\n--- Best per threshold mode (Strategy B) ---\n');
for m = ["raw","volscaled"]
rows = validRowsB & devTableB.Mode == m & devTableB.DDMult == 1;
if any(rows)
tmp = sortrows(devTableB(rows,:), 'SharpeNet', 'descend');
fprintf('%-12s ARMA(%d,%d) thr=%.5f Sharpe=%.4f
Trades=%.0f\n', ...
m, tmp.p(1), tmp.q(1), tmp.Threshold(1), tmp.SharpeNet(1),
tmp.NTrades(1));
else
fprintf('%-12s no valid configuration\n', m);
end
end
if cfg.useDoubleDown
fprintf('\n--- Best double-down configs (Strategy B) ---\n');
for mult = cfg.ddMultipliers
rows = validRowsB & devTableB.DDMult == mult;
if any(rows)
tmp = sortrows(devTableB(rows,:), 'SharpeNet', 'descend');
fprintf('DDMult=%.0fx ARMA(%d,%d) mode=%-10s thr1=%.5f
Sharpe=%.4f Trades=%.0f\n', ...
mult, tmp.p(1), tmp.q(1), tmp.Mode(1), tmp.Threshold(1),
tmp.SharpeNet(1), tmp.NTrades(1));


end
end
end
fprintf('\n========== SECTION 10: BEST DEV CONFIG — STRATEGY A vs STRATEGY B
==========\n');
validStdA = validRowsA & devTableA.DDMult == 1;
validStdB = validRowsB & devTableB.DDMult == 1;
bestPerModelA = bestConfigPerModel(devTableA, validStdA);
bestPerModelB = bestConfigPerModel(devTableB, validStdB);
fprintf('\nStrategy A — Plain ARMA (standard):\n');
disp(bestPerModelA(:,
{'p','q','Mode','Threshold','DDMult','SharpeNet','NTrades','FlatPct'}))
fprintf('Strategy B — ARMA-GARCH (standard):\n');
disp(bestPerModelB(:,
{'p','q','Mode','Threshold','DDMult','SharpeNet','NTrades','FlatPct'}))
if cfg.useDoubleDown
validDDA = validRowsA & devTableA.DDMult > 1;
validDDB = validRowsB & devTableB.DDMult > 1;
if any(validDDA)
fprintf('\nStrategy A — Plain ARMA (best double-down):\n');
tmpDD = sortrows(devTableA(validDDA,:), 'SharpeNet', 'descend');
disp(tmpDD(1:min(3,height(tmpDD)),
{'p','q','Mode','Threshold','DDMult','SharpeNet','NTrades','FlatPct'}))
end
if any(validDDB)
fprintf('Strategy B — ARMA-GARCH (best double-down):\n');
tmpDD = sortrows(devTableB(validDDB,:), 'SharpeNet', 'descend');
disp(tmpDD(1:min(3,height(tmpDD)),
{'p','q','Mode','Threshold','DDMult','SharpeNet','NTrades','FlatPct'}))
end
end
fprintf('Dev Buy-and-Hold Sharpe: %.4f\n', sBhDev.Sharpe);
fprintf('\n========== SECTION 11: PRIMARY MODEL SELECTION ==========\n');
bestRowA = selectBestWithParsimony(devTableA(validRowsA,:),
cfg.parsimonyTol);
bestRowB = selectBestWithParsimony(devTableB(validRowsB,:),
cfg.parsimonyTol);
fprintf('Strategy A (plain ARMA) : ARMA(%d,%d) mode=%-12s thr=%.5f
DDMult=%.0fx dev Sharpe=%.4f\n', ...
bestRowA.p, bestRowA.q, bestRowA.Mode, bestRowA.Threshold,
bestRowA.DDMult, bestRowA.SharpeNet);
fprintf('Strategy B (ARMA-GARCH) : ARMA(%d,%d) mode=%-12s thr=%.5f
```

```
DDMult=%.0fx dev Sharpe=%.4f\n', ...
bestRowB.p, bestRowB.q, bestRowB.Mode, bestRowB.Threshold,
bestRowB.DDMult, bestRowB.SharpeNet);
bestPA = bestRowA.p; bestQA = bestRowA.q;
bestModeA = bestRowA.Mode; bestThrA = bestRowA.Threshold; bestDDA =
bestRowA.DDMult;
bestPB = bestRowB.p; bestQB = bestRowB.q;


bestModeB = bestRowB.Mode; bestThrB = bestRowB.Threshold; bestDDB =
bestRowB.DDMult;
%% ============================================================
% 12. TAYLOR (2007) MOVING-AVERAGE RULE — STANDALONE ANALYSIS
%
Direct reproduction of the Script 1 methodology:
%
- 80/20 in-sample/out-of-sample split on the PRICE index
%
- movmean(Close,[S-1,0]) — MA includes the current close
%
- Three-state signal {-1, 0, +1} via bandwidth B
%
- Transaction cost charged on position CHANGES only
%
(no charge for the initial entry; positionChange = [0;
|diff(signal)|])
%
- Daily Sharpe = mean/std (no risk-free rate adjustment)
%
- Wealth curve = exp(cumsum(log-returns))
%
- Best (S,L,B) selected by net OOS Sharpe (data-snooped, as in Script
1)
% ============================================================
fprintf('\n========== SECTION 12: TAYLOR MA RULE (STANDALONE, SCRIPT 1 LOGIC)
==========\n');
Close
= price;
ret
= diff(log(Close));
nPriceMA
= length(Close);
oosStartPriceIdx = floor(0.80 * nPriceMA);
fprintf('OOS price index range : %d .. %d\n', oosStartPriceIdx, nPriceMA 1);
fprintf('OOS length
: %d trading days\n', nPriceMA oosStartPriceIdx);
fprintf('OOS start date
: %s\n', string(date(oosStartPriceIdx + 1)));
S_list = cfg.S_list;
L_list = cfg.L_list;
B_list = cfg.B_list;
sweepRows
= [];
bestSharpe_MA = -Inf;
bestPack_MA
= struct();
fprintf('\nParameter sweep (selecting best by net OOS Sharpe):\n');
fprintf(' %4s %4s %8s %14s %14s %10s\n', ...
'S', 'L', 'B', 'OOS Sharpe', 'Mean Net', 'NTrades');
fprintf(' ------------------------------------------------------------\n');
for S = S_list
for L = L_list
if S >= L
continue;
end
for B = B_list
% --- Script 1 signal construction --shortMA = movmean(Close, [S-1, 0]);
longMA = movmean(Close, [L-1, 0]);
R
= (shortMA - longMA) ./ longMA;
signal = nan(nPriceMA, 1);
for t = L:nPriceMA
if R(t) > B
signal(t) = 1;
elseif R(t) < -B


signal(t) = -1;
else
signal(t) = 0;
end
end
% --- Script 1 alignment with next-day returns --signalForRet = signal(1:end-1);
oosRetIdx
= oosStartPriceIdx:(length(Close) - 1);
validMask
= ~isnan(signalForRet(oosRetIdx));
oosRetIdx
= oosRetIdx(validMask);
oosSignal = signalForRet(oosRetIdx);
oosRet
= ret(oosRetIdx);
% --- Script 1 strategy returns and TC --stratRetGross = oosSignal .* oosRet;
positionChange = [0; abs(diff(oosSignal))];
stratRetNet
= stratRetGross - cfg.tc * positionChange;
meanNet = mean(stratRetNet);
volNet = std(stratRetNet);
if volNet > 0
sharpeNetMA = meanNet / volNet;
else
sharpeNetMA = NaN;
end
```

```
numTrades = sum(positionChange > 0);
sweepRows = [sweepRows; S, L, B, sharpeNetMA, meanNet,
numTrades]; %#ok<AGROW>
fprintf(' %4d %4d %8.4f %14.6f %14.6f %10d\n', ...
S, L, B, sharpeNetMA, meanNet, numTrades);
if isfinite(sharpeNetMA) && sharpeNetMA > bestSharpe_MA
bestSharpe_MA = sharpeNetMA;
bestPack_MA = struct( ...
'S', S, 'L', L, 'B', B, ...
'shortMA', shortMA, 'longMA', longMA, ...
'oosRetIdx', oosRetIdx, ...
'oosSignal', oosSignal, 'oosRet', oosRet, ...
'stratRetGross', stratRetGross, ...
'stratRetNet',
stratRetNet, ...
'positionChange', positionChange);
end
end
end
end
MASweepTable = array2table(sweepRows, ...
'VariableNames', {'S','L','B','SharpeNet','MeanNet','NumTrades'});
fprintf('\n');
disp(MASweepTable)
% --- Detailed reporting for the best (S, L, B) --bestS_MA = bestPack_MA.S;
bestL_MA = bestPack_MA.L;
bestB_MA = bestPack_MA.B;
oosSignal_MA
oosRet_MA


= bestPack_MA.oosSignal;
= bestPack_MA.oosRet;

stratRetGross_MA = bestPack_MA.stratRetGross;
stratRetNet_MA
= bestPack_MA.stratRetNet;
positionChange_MA = bestPack_MA.positionChange;
oosRetIdx_best
= bestPack_MA.oosRetIdx;
shortMA_best
= bestPack_MA.shortMA;
longMA_best
= bestPack_MA.longMA;
oosDates_MA
= date(oosRetIdx_best + 1);
buyHoldRet_MA
= oosRet_MA;
meanGrossMA = mean(stratRetGross_MA);
volGrossMA = std(stratRetGross_MA);
sharpeGrossMA = meanGrossMA / volGrossMA;
meanNetMA = mean(stratRetNet_MA);
volNetMA = std(stratRetNet_MA);
sharpeNetMA_best = meanNetMA / volNetMA;
annMeanGrossMA
= 252 * meanGrossMA;
annVolGrossMA
= sqrt(252) * volGrossMA;
annSharpeGrossMA = annMeanGrossMA / annVolGrossMA;
annMeanNetMA
= 252 * meanNetMA;
annVolNetMA
= sqrt(252) * volNetMA;
annSharpeNetMA = annMeanNetMA / annVolNetMA;
pnlGrossMA = sum(stratRetGross_MA);
pnlNetMA
= sum(stratRetNet_MA);
pnlBHMA
= sum(buyHoldRet_MA);
wealthGrossMA = exp(cumsum(stratRetGross_MA));
wealthNetMA
= exp(cumsum(stratRetNet_MA));
wealthBHMA
= exp(cumsum(buyHoldRet_MA));
numTradesMA
= sum(positionChange_MA > 0);
avgAbsPositionMA = mean(abs(oosSignal_MA));
fprintf('\nBest (S, L, B): S=%d, L=%d, B=%.4f
OOS Sharpe=%.4f\n', ...
bestS_MA, bestL_MA, bestB_MA, bestSharpe_MA);
fprintf('\n==========================================\n');
fprintf('TAYLOR MOVING-AVERAGE RULE (OUT OF SAMPLE)\n');
fprintf('==========================================\n');
fprintf('S = %d, L = %d, B = %.4f\n', bestS_MA, bestL_MA, bestB_MA);
fprintf('Transaction cost per unit change = %.4f\n', cfg.tc);
fprintf('OOS observations
= %d\n', length(oosRet_MA));
fprintf('Number of trades
= %d\n', numTradesMA);
fprintf('Average absolute position = %.4f\n\n', avgAbsPositionMA);
fprintf('Gross performance:\n');
fprintf(' Mean daily return
fprintf(' Daily volatility
fprintf(' Daily Sharpe ratio
fprintf(' Annualized mean
fprintf(' Annualized volatility
fprintf(' Annualized Sharpe
fprintf(' Cumulative P/L (log)

= %.6f\n', meanGrossMA);
= %.6f\n', volGrossMA);
= %.6f\n', sharpeGrossMA);
= %.6f\n', annMeanGrossMA);
```

```
= %.6f\n', annVolGrossMA);
= %.6f\n', annSharpeGrossMA);
= %.6f\n\n', pnlGrossMA);

fprintf('Net performance:\n');
fprintf(' Mean daily return
fprintf(' Daily volatility
fprintf(' Daily Sharpe ratio

= %.6f\n', meanNetMA);
= %.6f\n', volNetMA);
= %.6f\n', sharpeNetMA_best);


fprintf('
fprintf('
fprintf('
fprintf('

Annualized mean
Annualized volatility
Annualized Sharpe
Cumulative P/L (log)

= %.6f\n', annMeanNetMA);
= %.6f\n', annVolNetMA);
= %.6f\n', annSharpeNetMA);
= %.6f\n\n', pnlNetMA);

fprintf('Buy-and-hold benchmark:\n');
fprintf(' Mean daily return
= %.6f\n', mean(buyHoldRet_MA));
fprintf(' Daily volatility
= %.6f\n', std(buyHoldRet_MA));
fprintf(' Daily Sharpe ratio
= %.6f\n',
mean(buyHoldRet_MA)/std(buyHoldRet_MA));
fprintf(' Cumulative P/L (log)
= %.6f\n', pnlBHMA);
ResultsMA = table( ...
meanGrossMA, volGrossMA, sharpeGrossMA, pnlGrossMA, ...
meanNetMA,
volNetMA,
sharpeNetMA_best, pnlNetMA, ...
numTradesMA, ...
'VariableNames', { ...
'MeanGross','VolGross','SharpeGross','PnLGross', ...
'MeanNet','VolNet','SharpeNet','PnLNet', ...
'NumTrades'});
fprintf('\n');
disp(ResultsMA)
% --- Figures (Script 1 style) --figure('Name','MA Trading Rule — Price + Moving
Averages','NumberTitle','off');
plot(date, Close,
'k-', 'LineWidth', 1);
hold on;
plot(date, shortMA_best, 'b-', 'LineWidth', 1.2);
plot(date, longMA_best, 'r-', 'LineWidth', 1.2);
xline(date(oosStartPriceIdx), 'k:', 'LineWidth', 1, 'Label', 'OOS start');
legend('Close', ...
sprintf('Short MA (S=%d)', bestS_MA), ...
sprintf('Long MA (L=%d)', bestL_MA), ...
'Location','best');
title(sprintf('Taylor MA Rule: S=%d, L=%d, B=%.4f', bestS_MA, bestL_MA,
bestB_MA));
xlabel('Date'); ylabel('Price'); grid on; set(gca,'FontSize',11);
print('fig_ma_price_with_mas','-dpng','-r300');
figure('Name','MA Trading Rule — OOS Cumulative Wealth','NumberTitle','off');
plot(oosDates_MA, wealthGrossMA, 'b-', 'LineWidth', 1.5); hold on;
plot(oosDates_MA, wealthNetMA,
'r--', 'LineWidth', 1.5);
plot(oosDates_MA, wealthBHMA,
'k:', 'LineWidth', 1.5);
yline(1, 'Color', [0.6 0.6 0.6], 'LineStyle', ':');
legend('MA Gross','MA Net','Buy & Hold','Location','best');
title(sprintf('Out-of-Sample Performance: S=%d, L=%d, B=%.4f', bestS_MA,
bestL_MA, bestB_MA));
xlabel('Date'); ylabel('Wealth (exp(cumsum log-returns)))');
grid on; set(gca,'FontSize',11);
print('fig_ma_oos_wealth','-dpng','-r300');
fprintf('\n========== SECTION 13: TEST-PERIOD SIGNAL GENERATION
==========\n');
testRet
= simpRet(idx.testStart:idx.testEnd);
datesTest = retDate(idx.testStart:idx.testEnd);
% --- Strategy A: plain ARMA --[fcA, condVolA] = rollingArmaForecasts( ...
logRet, idx.testStart, idx.testEnd, cfg.winLen, bestPA, bestQA, ...
cfg.refitEvery, false);


sigARaw = buildSignalFromMode(fcA, condVolA, bestModeA, bestThrA,
cfg.pctLookback, bestDDA);
sigA
= applyMinHoldingPeriod(sigARaw, cfg.minHold);
fprintf('Strategy A — ARMA(%d,%d) mode=%s thr=%.5f DDMult=%.0fx\n', ...
bestPA, bestQA, bestModeA, bestThrA, bestDDA);
fprintf(' Pos +2: %.1f%% +1: %.1f%% 0: %.1f%% -1: %.1f%% 2: %.1f%%\n', ...
100*mean(sigA==2), 100*mean(sigA==1), 100*mean(sigA==0), ...
100*mean(sigA==-1), 100*mean(sigA==-2));
fprintf(' Forecast: mean=%.6f std=%.6f skew=%.4f +ve=%.1f%% ve=%.1f%%\n', ...
mean(fcA), std(fcA), skewness(fcA), 100*mean(fcA>0), 100*mean(fcA<0));
% --- Strategy B: ARMA-GARCH --[fcB, condVolB] = rollingArmaForecasts( ...
logRet, idx.testStart, idx.testEnd, cfg.winLen, bestPB, bestQB, ...
```

```
cfg.refitEvery, true);
sigBRaw = buildSignalFromMode(fcB, condVolB, bestModeB, bestThrB,
cfg.pctLookback, bestDDB);
if shouldApplyGarchScaling(cfgB, bestModeB)
sigBRaw = applyGarchScalingDD(sigBRaw, condVolB, cfg.sigmaTarget);
end
sigB = applyMinHoldingPeriod(sigBRaw, cfg.minHold);
fprintf('\nStrategy B — ARMA(%d,%d)-GARCH(1,1) mode=%s thr=%.5f
DDMult=%.0fx\n', ...
bestPB, bestQB, bestModeB, bestThrB, bestDDB);
fprintf(' Pos +2: %.1f%% +1: %.1f%% 0: %.1f%% -1: %.1f%% 2: %.1f%%\n', ...
100*mean(sigB==2), 100*mean(sigB==1), 100*mean(sigB==0), ...
100*mean(sigB==-1), 100*mean(sigB==-2));
fprintf(' Forecast: mean=%.6f std=%.6f skew=%.4f +ve=%.1f%% ve=%.1f%%\n', ...
mean(fcB), std(fcB), skewness(fcB), 100*mean(fcB>0), 100*mean(fcB<0));
% --- MA signal on the ARMA test sample (uses bestS_MA, bestL_MA, bestB_MA
from Section 12) --maSignal = buildMASignal(price, idx.testStart, idx.testEnd, ...
bestS_MA, bestL_MA, bestB_MA);
fprintf('\nMA(S=%d, L=%d, B=%.4f): Long: %.1f%%
Short: %.1f%%
Flat: %.1f%%\n', ...
bestS_MA, bestL_MA, bestB_MA, ...
100*mean(maSignal==1), 100*mean(maSignal==-1), 100*mean(maSignal==0));
%% ============================================================
% TEST-PERIOD PERFORMANCE TABLE
% ============================================================
fprintf('\n========== SECTION 14: TEST-PERIOD PERFORMANCE ==========\n');
retAGross = sigA .* testRet;
retBGross = sigB .* testRet;
retMaGross = maSignal .* testRet;
bhRet
= testRet;
tradesA = tradeUnits(sigA);
tradesB = tradeUnits(sigB);
tradesMa = tradeUnits(maSignal);
tradesBH = zeros(T_test,1);


retANet = retAGross - cfg.tc * tradesA;
retBNet = retBGross - cfg.tc * tradesB;
retMaNet = retMaGross - cfg.tc * tradesMa;
sAGross = summariseReturns(retAGross, tradesA, cfg.rfDaily);
sANet
= summariseReturns(retANet,
tradesA, cfg.rfDaily);
sBGross = summariseReturns(retBGross, tradesB, cfg.rfDaily);
sBNet
= summariseReturns(retBNet,
tradesB, cfg.rfDaily);
sMaGross = summariseReturns(retMaGross, tradesMa, cfg.rfDaily);
sMaNet
= summariseReturns(retMaNet,
tradesMa, cfg.rfDaily);
sBH
= summariseReturns(bhRet,
tradesBH, cfg.rfDaily);
ddAGross = maxDrawdown(retAGross);
ddANet
= maxDrawdown(retANet);
ddBGross = maxDrawdown(retBGross);
ddBNet
= maxDrawdown(retBNet);
ddMaGross = maxDrawdown(retMaGross);
ddMaNet
= maxDrawdown(retMaNet);
ddBH
= maxDrawdown(bhRet);
results = table( ...
{'ARMA (gross)';'ARMA (net)';'ARMA-GARCH (gross)';'ARMA-GARCH (net)'; ...
'MA (gross)';'MA (net)';'Buy & Hold'}, ...
round([sAGross.CumReturn; sANet.CumReturn; sBGross.CumReturn;
sBNet.CumReturn; ...
sMaGross.CumReturn; sMaNet.CumReturn; sBH.CumReturn], 4), ...
round([sAGross.AnnReturn; sANet.AnnReturn; sBGross.AnnReturn;
sBNet.AnnReturn; ...
sMaGross.AnnReturn; sMaNet.AnnReturn; sBH.AnnReturn], 4), ...
round([sAGross.AnnVol; sANet.AnnVol; sBGross.AnnVol; sBNet.AnnVol; ...
sMaGross.AnnVol; sMaNet.AnnVol; sBH.AnnVol], 4), ...
round([sAGross.Sharpe; sANet.Sharpe; sBGross.Sharpe; sBNet.Sharpe; ...
sMaGross.Sharpe; sMaNet.Sharpe; sBH.Sharpe], 4), ...
round([sAGross.NTrades; sANet.NTrades; sBGross.NTrades;
sBNet.NTrades; ...
sMaGross.NTrades; sMaNet.NTrades; sBH.NTrades], 1), ...
round([ddAGross; ddANet; ddBGross; ddBNet; ddMaGross; ddMaNet; ddBH],
4), ...
'VariableNames',
{'Strategy','Cum_Return','Ann_Return','Ann_Vol','Sharpe','N_Trades','Max_DD'}
);
disp(results)
fprintf('\n========== SECTION 15: FORECAST DIRECTIONAL ACCURACY
==========\n');
actualDir = sign(testRet);
hitA = mean(sign(fcA) == actualDir);
hitB = mean(sign(fcB) == actualDir);
fprintf('Strategy A (plain ARMA):\n');
fprintf(' Overall hit rate
: %.2f%%\n', 100*hitA);
nzA = sign(fcA) ~= 0;
if any(nzA)
fprintf(' Active forecast hit rate : %.2f%%\n', ...
100*mean(sign(fcA(nzA)) == actualDir(nzA)));
end
nzSigA = sigA ~= 0;
```

```
if any(nzSigA)


fprintf(' Active signal hit rate
: %.2f%%\n', ...
100*mean(sign(sigA(nzSigA)) == actualDir(nzSigA)));
end
fprintf('Strategy B (ARMA-GARCH):\n');
fprintf(' Overall hit rate
: %.2f%%\n', 100*hitB);
nzB = sign(fcB) ~= 0;
if any(nzB)
fprintf(' Active forecast hit rate : %.2f%%\n', ...
100*mean(sign(fcB(nzB)) == actualDir(nzB)));
end
nzSigB = sigB ~= 0;
if any(nzSigB)
fprintf(' Active signal hit rate
: %.2f%%\n', ...
100*mean(sign(sigB(nzSigB)) == actualDir(nzSigB)));
end
fprintf('\n========== SECTION 16: YEAR-BY-YEAR RETURN BREAKDOWN
==========\n');
years
= year(datesTest);
uniqueYears = unique(years)';
fprintf('%-6s %-14s %-16s %-14s %-10s\n', ...
'Year','ARMA(net)','ARMA-GARCH(net)','MA(net)','Buy&Hold');
fprintf('%s\n', repmat('-',1,64));
for yr = uniqueYears
mask = years == yr;
nObs = sum(mask);
annA = mean(retANet(mask)) * 252 * 100;
annB = mean(retBNet(mask)) * 252 * 100;
annMa = mean(retMaNet(mask)) * 252 * 100;
annBH = mean(bhRet(mask))
* 252 * 100;
if nObs < 5
fprintf('%-6d %-14.2f %-16.2f %-14.2f %-10.2f [n=%d, treat with
caution]\n', ...
yr, annA, annB, annMa, annBH, nObs);
else
fprintf('%-6d %-14.2f %-16.2f %-14.2f %-10.2f\n', yr, annA, annB,
annMa, annBH);
end
end
fprintf('\n========== SECTION 17: HAC MEAN RETURN DIFFERENTIAL TESTS
==========\n');
fprintf('H0: equal mean net returns (two-sided, HAC standard errors)\n\n');
[dmAB,
pAB]
= hacMeanDiffTest(retANet - retBNet);
[dmAMa, pAMa] = hacMeanDiffTest(retANet - retMaNet);
[dmABH, pABH] = hacMeanDiffTest(retANet - bhRet);
[dmBMa, pBMa] = hacMeanDiffTest(retBNet - retMaNet);
[dmBBH, pBBH] = hacMeanDiffTest(retBNet - bhRet);
[dmMaBH, pMaBH] = hacMeanDiffTest(retMaNet - bhRet);
fprintf('ARMA vs ARMA-GARCH : stat=%7.4f
fprintf('ARMA vs MA
: stat=%7.4f
fprintf('ARMA vs B&H
: stat=%7.4f
fprintf('ARMA-GARCH vs MA
: stat=%7.4f
fprintf('ARMA-GARCH vs B&H : stat=%7.4f
fprintf('MA vs B&H
: stat=%7.4f


p=%.4f\n', dmAB,
pAB);
p=%.4f\n', dmAMa, pAMa);
p=%.4f\n', dmABH, pABH);
p=%.4f\n', dmBMa, pBMa);
p=%.4f\n', dmBBH, pBBH);
p=%.4f\n', dmMaBH, pMaBH);

fprintf('\n========== SECTION 18: LEDOIT-WOLF SHARPE TESTS ==========\n');
fprintf('H0: equal Sharpe ratio (two-sided)\n\n');
[lwAB,
plwAB]
= ledoitWolfSharpeTest(retANet, retBNet);
[lwAMa, plwAMa] = ledoitWolfSharpeTest(retANet, retMaNet);
[lwABH, plwABH] = ledoitWolfSharpeTest(retANet, bhRet);
[lwBMa, plwBMa] = ledoitWolfSharpeTest(retBNet, retMaNet);
[lwBBH, plwBBH] = ledoitWolfSharpeTest(retBNet, bhRet);
[lwMaBH, plwMaBH] = ledoitWolfSharpeTest(retMaNet, bhRet);
fprintf('ARMA vs ARMA-GARCH : stat=%7.4f
fprintf('ARMA vs MA
: stat=%7.4f
fprintf('ARMA vs B&H
: stat=%7.4f
fprintf('ARMA-GARCH vs MA
: stat=%7.4f
fprintf('ARMA-GARCH vs B&H : stat=%7.4f
fprintf('MA vs B&H
: stat=%7.4f

p=%.4f\n', lwAB,
plwAB);
p=%.4f\n', lwAMa, plwAMa);
p=%.4f\n', lwABH, plwABH);
p=%.4f\n', lwBMa, plwBMa);
```

```
p=%.4f\n', lwBBH, plwBBH);
p=%.4f\n', lwMaBH, plwMaBH);

fprintf('\n========== SECTION 19: TRANSACTION-COST SENSITIVITY
==========\n');
fprintf('%-10s %-12s %-14s %-12s %-12s\n', ...
'TC (bps)','ARMA Sharpe','ARMA-G Sharpe','MA Sharpe','B&H Sharpe');
fprintf('%s\n', repmat('-',1,64));
tcSens = zeros(length(cfg.tcGrid), 4);
for j = 1:length(cfg.tcGrid)
tcj = cfg.tcGrid(j);
sA_ = summariseReturns(retAGross - tcj*tradesA, tradesA,
cfg.rfDaily);
sB_ = summariseReturns(retBGross - tcj*tradesB, tradesB,
cfg.rfDaily);
sMa_ = summariseReturns(retMaGross - tcj*tradesMa, tradesMa,
cfg.rfDaily);
tcSens(j,:) = [sA_.Sharpe, sB_.Sharpe, sMa_.Sharpe, sBH.Sharpe];
fprintf('%-10.1f %-12.4f %-14.4f %-12.4f %-12.4f\n', ...
tcj*10000, sA_.Sharpe, sB_.Sharpe, sMa_.Sharpe, sBH.Sharpe);
end
fprintf('\nApproximate break-even transaction costs:\n');
if sum(tradesA) > 0 && sAGross.CumReturn > 0
try
beA = fzero(@(tc) prod(1 + retAGross - tc*tradesA) - 1, [0, 1],
optimset('Display','off'));
fprintf(' ARMA
: %.4f (%.1f bps)\n', beA, beA*10000);
catch
fprintf(' ARMA
: N/A\n');
end
else
fprintf(' ARMA
: N/A (gross return is negative — no break-even
exists)\n');
end
if sum(tradesB) > 0 && sBGross.CumReturn > 0
try
beB = fzero(@(tc) prod(1 + retBGross - tc*tradesB) - 1, [0, 1],
optimset('Display','off'));
fprintf(' ARMA-GARCH : %.4f (%.1f bps)\n', beB, beB*10000);
catch
fprintf(' ARMA-GARCH : N/A\n');
end
else
fprintf(' ARMA-GARCH : N/A (gross return is negative — no break-even
exists)\n');


end
if sum(tradesMa) > 0 && sMaGross.CumReturn > 0
try
beMa = fzero(@(tc) prod(1 + retMaGross - tc*tradesMa) - 1, [0, 1],
optimset('Display','off'));
fprintf(' MA
: %.4f (%.1f bps)\n', beMa, beMa*10000);
catch
fprintf(' MA
: N/A\n');
end
else
fprintf(' MA
: N/A (gross return is negative — no break-even
exists)\n');
end
%% ========================================================
% 20. CONSOLIDATED SUMMARY
% ========================================================
fprintf('\n======================================================================\n
');
fprintf('
CONSOLIDATED SUMMARY\n');
fprintf('======================================================================\n')
;
fprintf('Strategy A : ARMA(%d,%d)
mode=%-12s thr=%.5f
DDMult=%.0fx\n', ...
bestPA, bestQA, bestModeA, bestThrA, bestDDA);
fprintf('Strategy B : ARMA(%d,%d)-GARCH(1,1) mode=%-12s thr=%.5f
DDMult=%.0fx\n', ...
bestPB, bestQB, bestModeB, bestThrB, bestDDB);
fprintf('Benchmark : MA(S=%d, L=%d, B=%.4f) [Script 1 logic; OOS
Sharpe=%.4f]\n', ...
bestS_MA, bestL_MA, bestB_MA, bestSharpe_MA);
fprintf('Holding period
: %d days\n',
cfg.minHold);
fprintf('Transaction cost : %.1f bps per trade unit\n', cfg.tc*10000);
fprintf('Refit frequency : %d days\n',
cfg.refitEvery);
fprintf('Risk-free rate
: %.2f%% p.a.\n', cfg.rfAnnual*100);
fprintf('Note: Ann. return = mean*252 (arithmetic); Cum. return =
geometric.\n');
fprintf('Note: TC scales with position-size change (tradeUnits = abs(pos_t pos_{t-1})).\n');
fprintf('----------------------------------------------------------------------\n');
fprintf('%-22s %-12s %-14s %-12s %-10s\n', ...
'', 'ARMA(net)', 'ARMA-G(net)', 'MA(net)', 'BuyHold');
fprintf('----------------------------------------------------------------------\n');
fprintf('%-22s %-12.4f %-14.4f %-12.4f %-10.4f\n', 'OOS/Dev Sharpe', ...
bestRowA.SharpeNet, bestRowB.SharpeNet, bestSharpe_MA, sBhDev.Sharpe);
fprintf('%-22s %-12.4f %-14.4f %-12.4f %-10.4f\n', 'Test Cum Return', ...
```

```
sANet.CumReturn, sBNet.CumReturn, sMaNet.CumReturn, sBH.CumReturn);
fprintf('%-22s %-12.4f %-14.4f %-12.4f %-10.4f\n', 'Test Ann Return', ...
sANet.AnnReturn, sBNet.AnnReturn, sMaNet.AnnReturn, sBH.AnnReturn);
fprintf('%-22s %-12.4f %-14.4f %-12.4f %-10.4f\n', 'Test Ann Vol', ...
sANet.AnnVol, sBNet.AnnVol, sMaNet.AnnVol, sBH.AnnVol);
fprintf('%-22s %-12.4f %-14.4f %-12.4f %-10.4f\n', 'Test Sharpe', ...
sANet.Sharpe, sBNet.Sharpe, sMaNet.Sharpe, sBH.Sharpe);
fprintf('%-22s %-12.0f %-14.0f %-12.0f %-10.0f\n', 'N Trade Units', ...
sANet.NTrades, sBNet.NTrades, sMaNet.NTrades, sBH.NTrades);
fprintf('%-22s %-12.4f %-14.4f %-12.4f %-10.4f\n', 'Max Drawdown
(net)', ...
ddANet, ddBNet, ddMaNet, ddBH);
fprintf('%-22s %-12.2f %-14.2f %-12s %-10s\n', 'Hit Rate (%)', ...


100*hitA, 100*hitB, 'N/A', 'N/A');
fprintf('----------------------------------------------------------------\n');
fprintf('HAC ARMA vs ARMA-G : stat=%7.4f p=%.4f\n', dmAB,
pAB);
fprintf('HAC ARMA vs MA
: stat=%7.4f p=%.4f\n', dmAMa, pAMa);
fprintf('HAC ARMA vs B&H
: stat=%7.4f p=%.4f\n', dmABH, pABH);
fprintf('HAC ARMA-G vs MA
: stat=%7.4f p=%.4f\n', dmBMa, pBMa);
fprintf('HAC ARMA-G vs B&H
: stat=%7.4f p=%.4f\n', dmBBH, pBBH);
fprintf('HAC MA vs B&H
: stat=%7.4f p=%.4f\n', dmMaBH, pMaBH);
fprintf('LW
ARMA vs ARMA-G : stat=%7.4f p=%.4f\n', lwAB,
plwAB);
fprintf('LW
ARMA vs MA
: stat=%7.4f p=%.4f\n', lwAMa, plwAMa);
fprintf('LW
ARMA vs B&H
: stat=%7.4f p=%.4f\n', lwABH, plwABH);
fprintf('LW
ARMA-G vs MA
: stat=%7.4f p=%.4f\n', lwBMa, plwBMa);
fprintf('LW
ARMA-G vs B&H
: stat=%7.4f p=%.4f\n', lwBBH, plwBBH);
fprintf('LW
MA vs B&H
: stat=%7.4f p=%.4f\n', lwMaBH, plwMaBH);
fprintf('================================================================\n')
;
%% ============================================================
% FIGURES (TEST-SAMPLE COMPARISON)
% ============================================================
figure('Name','Figure 1: Cumulative Wealth','NumberTitle','off');
plot(datesTest, cumprod(1+retANet), 'b-', 'LineWidth', 1.5); hold on;
plot(datesTest, cumprod(1+retBNet), 'g--', 'LineWidth', 1.5);
plot(datesTest, cumprod(1+retMaNet), 'r-.', 'LineWidth', 1.5);
plot(datesTest, cumprod(1+bhRet),
'k:', 'LineWidth', 1.5);
yline(1, 'Color', [0.6 0.6 0.6], 'LineStyle', ':', 'LineWidth', 0.8);
legend(sprintf('ARMA(%d,%d) net (DD=%.0fx)', bestPA, bestQA, bestDDA), ...
sprintf('ARMA(%d,%d)-GARCH(1,1) net (DD=%.0fx)', bestPB, bestQB,
bestDDB), ...
sprintf('MA(S=%d,L=%d,B=%.4f) net', bestS_MA, bestL_MA, bestB_MA), ...
'Buy & Hold', 'Location', 'northwest');
xlabel('Date'); ylabel('Wealth (normalised to 1)');
title(sprintf('Figure 1: Cumulative Wealth — Test Sample (TC = %.0f
bps/unit)', cfg.tc*10000));
grid on; set(gca,'FontSize',11);
print('fig1_cumulative_wealth','-dpng','-r300');
figure('Name','Figure 2: Signals and Conditional
Volatility','NumberTitle','off');
subplot(3,1,1);
priceEnd = min(idx.testEnd + 1, length(price));
plot(datesTest, price(idx.testStart+1:priceEnd), 'k-', 'LineWidth', 1);
ylabel('Price (USD)'); grid on;
title('Figure 2: Strategy Signals — Test Sample');
subplot(3,1,2);
plot(datesTest, condVolB * sqrt(252) * 100, 'Color', [0.6 0 0.8],
'LineWidth', 1);
ylabel('GARCH Cond. Vol (% ann.)'); grid on;
subplot(3,1,3);
stairs(datesTest, sigB, 'Color', [0 0.45 0.74], 'LineWidth', 1.2); hold on;
stairs(datesTest, sigA, 'Color', [0.85 0.33 0.10], 'LineWidth', 1.0,
'LineStyle', '--');
yline(0, 'k:', 'LineWidth', 0.8);
ylim([-2.5 2.5]); yticks([-2 -1 0 1 2]);
yticklabels({'Dbl Short','Short','Flat','Long','Dbl Long'});
legend('ARMA-GARCH signal','ARMA signal','Location','southeast');
xlabel('Date'); ylabel('Position'); grid on;


set(gcf,'Position',[100 100 900 600]);
print('fig2_signals_condvol','-dpng','-r300');
figure('Name','Figure 3: TC Sensitivity','NumberTitle','off');
tcBps = cfg.tcGrid * 10000;
plot(tcBps, tcSens(:,1), 'b-o', 'LineWidth',1.5,'MarkerSize',5); hold on;
plot(tcBps, tcSens(:,2), 'g--^', 'LineWidth',1.5,'MarkerSize',5);
plot(tcBps, tcSens(:,3), 'r-.s', 'LineWidth',1.5,'MarkerSize',5);
plot(tcBps, tcSens(:,4), 'k:d', 'LineWidth',1.5,'MarkerSize',5);
yline(0,'Color',[0.6 0.6 0.6],'LineStyle','--');
legend(sprintf('ARMA(%d,%d) DD=%.0fx', bestPA, bestQA, bestDDA), ...
sprintf('ARMA(%d,%d)-GARCH DD=%.0fx', bestPB, bestQB, bestDDB), ...
sprintf('MA(S=%d,L=%d,B=%.4f)', bestS_MA, bestL_MA, bestB_MA), ...
```

```
'Buy & Hold','Location','northeast');
xlabel('Transaction Cost (bps per unit)'); ylabel('Annualised Sharpe Ratio');
title('Figure 3: Sharpe Ratio vs Transaction Cost — Test Sample');
grid on; set(gca,'FontSize',11);
print('fig3_tc_sensitivity','-dpng','-r300');
fprintf('\nFigures saved: fig_ma_price_with_mas.png fig_ma_oos_wealth.png
fig1_cumulative_wealth.png fig2_signals_condvol.png
fig3_tc_sensitivity.png\n');
%% ============================================================
% LOCAL FUNCTIONS
% ============================================================
function icTable = buildICTable(trainY, maxP, maxQ)
rows = [];
for p = 0:maxP
for q = 0:maxQ
if p==0 && q==0; continue; end
try
mdl = arima(p,0,q);
[~, ~, logL] = estimate(mdl, trainY, 'Display','off');
nObs = length(trainY); nParams = p+q+2;
aicVal = -2*logL + 2*nParams;
bicVal = -2*logL + nParams*log(nObs);
hqicVal = -2*logL + 2*nParams*log(log(nObs));
rows = [rows; p,q,aicVal,bicVal,hqicVal,nParams]; %#ok<AGROW>
fprintf(' ARMA(%d,%d): AIC=%.2f BIC=%.2f
HQIC=%.2f\n',p,q,aicVal,bicVal,hqicVal);
catch
fprintf(' ARMA(%d,%d): failed — skipped\n',p,q);
end
end
end
icTable =
array2table(rows,'VariableNames',{'p','q','AIC','BIC','HQIC','nParams'});
end
% ------------------------------------------------------------function [fc, condVol] =
rollingArmaForecasts( ...
logRet, startIdx, endIdx, winLen, p, q, refitEvery, useGarch)
if nargin < 8; useGarch = false; end
n = endIdx-startIdx+1; fc = zeros(n,1); condVol = zeros(n,1);
currentMeanFit = []; currentVarFit = [];
lastGoodFc = 0; lastGoodVol = std(logRet(startIdx-winLen:startIdx-1));
for i = 1:n
t = startIdx+i-1; y = logRet(t-winLen:t-1);
if isempty(currentMeanFit) || mod(i-1,refitEvery)==0


if useGarch
try
mdl = arima(p,0,q); mdl.Variance = garch(1,1);
[currentMeanFit,~,currentVarFit] =
estimate(mdl,y,'Display','off');
catch
currentMeanFit=[]; currentVarFit=[];
try; currentMeanFit=estimate(arima(p,0,q),y,'Display','off');
catch; end
end
else
try; currentMeanFit=estimate(arima(p,0,q),y,'Display','off');
catch; currentMeanFit=[]; end
end
end
if ~isempty(currentMeanFit)
try; fc(i)=forecast(currentMeanFit,1,'Y0',y); lastGoodFc=fc(i);
catch; fc(i)=lastGoodFc; end
else; fc(i)=0; end
if useGarch && ~isempty(currentMeanFit) && ~isempty(currentVarFit)
try
[resid,V,~] = infer(currentMeanFit,y);
gP = currentMeanFit.Variance;
h_next = gP.Constant + gP.ARCH{1}*resid(end)^2 +
gP.GARCH{1}*V(end);
condVol(i) = sqrt(max(h_next,eps)); lastGoodVol = condVol(i);
catch; condVol(i)=lastGoodVol; end
else; condVol(i)=max(std(y),eps); lastGoodVol=condVol(i); end
end
end
% ------------------------------------------------------------function signal = makeSignalsRaw(forecastVec,
thr1, thr2)
signal = zeros(length(forecastVec), 1);
signal(forecastVec > thr1) = 1;
signal(forecastVec < -thr1) = -1;
if nargin == 3 && ~isempty(thr2) && thr2 > thr1
signal(forecastVec > thr2) = 2;
signal(forecastVec < -thr2) = -2;
end
end
% ------------------------------------------------------------function scaledSignal =
applyGarchScalingDD(signal, condVol, sigmaTarget)
raw = signal .* (sigmaTarget ./ max(condVol, eps));
scaledSignal = max(min(round(raw), 2), -2);
end
% ------------------------------------------------------------function result = shouldApplyGarchScaling(cfg,
mode)
result = cfg.useGarch && cfg.garchScale && any(cfg.garchScaleModes == mode);
end
% ------------------------------------------------------------function sigRaw = buildSignalFromMode(fc,
condVol, mode, thr1, pctLookback,
ddMult)
if nargin < 6; ddMult = 1; end
thr2 = [];
if ddMult > 1
thr2 = ddMult * thr1;
end
```

```
if mode == "raw"
sigRaw = makeSignalsRaw(fc, thr1, thr2);
elseif mode == "volscaled"
sigRaw = makeSignalsRaw(fc ./ max(condVol, eps), thr1, thr2);
elseif mode == "percentile"
pctT = precomputePercentileThresholds(abs(fc), pctLookback, thr1);
sigRaw = makeSignalsPercentilePrecomputed(fc, pctT(:,1));
else
error('Unknown mode: %s', mode);
end
end
% ------------------------------------------------------------function pctThresholds =
precomputePercentileThresholds(absFcVec, lookback,
pctLevels)
n=length(absFcVec); nLevels=length(pctLevels);
pctThresholds=zeros(n,nLevels);
for i=1:n
if i==1; pctThresholds(i,:)=Inf;
else; s=max(1,i-lookback); pctThresholds(i,:)=prctile(absFcVec(s:i1),pctLevels*100); end
end
end
% ------------------------------------------------------------function signal =
makeSignalsPercentilePrecomputed(forecastVec,
dynThresholdVec)
signal=zeros(length(forecastVec),1); absFc=abs(forecastVec);
signal(absFc>dynThresholdVec & forecastVec>0)= 1;
signal(absFc>dynThresholdVec & forecastVec<0)=-1;
end
% ------------------------------------------------------------function signal =
applyMinHoldingPeriod(rawSignal, minHold)
signal = rawSignal;
n = length(signal);
if n <= 1 || minHold <= 1; return; end
holdCount = 1;
for i = 2:n
if sign(signal(i)) ~= sign(signal(i-1))
if holdCount >= minHold
holdCount = 1;
else
signal(i) = signal(i-1);
holdCount = holdCount + 1;
end
else
holdCount = holdCount + 1;
end
end
end
% ------------------------------------------------------------function trades = tradeUnits(signal)
trades = zeros(length(signal), 1);
if isempty(signal); return; end
trades(1) = abs(signal(1));
for i = 2:length(signal)
trades(i) = abs(signal(i) - signal(i-1));
end
end


% ------------------------------------------------------------function s = summariseReturns(retVec, trades,
rfDaily)
if nargin < 3; rfDaily = 0; end
cumRet = prod(1+retVec)-1; annRet = mean(retVec)*252;
annVol = std(retVec)*sqrt(252); sr = NaN;
if annVol > 0
sr = (mean(retVec) - rfDaily) / std(retVec) * sqrt(252);
end
s = struct('CumReturn',cumRet,'AnnReturn',annRet,'AnnVol',annVol, ...
'Sharpe',sr,'NTrades',sum(trades));
end
% ------------------------------------------------------------function mdd = maxDrawdown(retVec)
wealth = cumprod(1+retVec);
dd = (cummax(wealth) - wealth) ./ cummax(wealth);
mdd = max(dd);
end
% ------------------------------------------------------------function [stat, pVal] = hacMeanDiffTest(d)
n=length(d); dBar=mean(d); maxLag=floor(n^(1/3)); gamma0=var(d); gammaSum=0;
for j=1:maxLag
w=1-j/(maxLag+1); autoC=0;
for t=j+1:n; autoC=autoC+(d(t)-dBar)*(d(t-j)-dBar); end
gammaSum=gammaSum+2*w*(autoC/n);
end
hacVar=(gamma0+gammaSum)/n;
if hacVar>0; stat=dBar/sqrt(hacVar); pVal=2*(1-normcdf(abs(stat)));
else; stat=NaN; pVal=NaN; end
end
% ------------------------------------------------------------function [lwStat, pVal] =
ledoitWolfSharpeTest(ret1, ret2)
n=length(ret1); mu1=mean(ret1); mu2=mean(ret2); s1=std(ret1); s2=std(ret2);
if s1==0||s2==0; lwStat=NaN; pVal=NaN; return; end
sr1=mu1/s1; sr2=mu2/s2;
psi=(ret1-mu1)/s1-(sr1/2)*((ret1-mu1).^2/s1^2-1) ...
-(ret2-mu2)/s2+(sr2/2)*((ret2-mu2).^2/s2^2-1);
psiBar=mean(psi); maxLag=floor(n^(1/3)); gamma0=var(psi); gammaSum=0;
for j=1:maxLag
w=1-j/(maxLag+1); autoC=0;
for t=j+1:n; autoC=autoC+(psi(t)-psiBar)*(psi(t-j)-psiBar); end
gammaSum=gammaSum+2*w*(autoC/n);
end
hacVar=(gamma0+gammaSum)/n;
if hacVar>0; lwStat=(sr1-sr2)/sqrt(hacVar); pVal=2*(1-normcdf(abs(lwStat)));
else; lwStat=NaN; pVal=NaN; end
end
```

```
% ------------------------------------------------------------function [sNet, isTrivial, flatPct, longPct,
shortPct] = evaluateSignal( ...
signal, retVec, tc, rfDaily, minTrades, requireBothSides, ...
minLongPct, minShortPct, maxTradePct)
trades = tradeUnits(signal);
retNet = signal .* retVec - tc * trades;
sNet
= summariseReturns(retNet, trades, rfDaily);
dir
= sign(signal);
flatPct = 100*mean(dir==0);
longPct = 100*mean(dir>0);


shortPct = 100*mean(dir<0);
tradePct = 100*sum(trades)/length(signal);
oneSided = (all(dir>=0) && any(dir>0)) || (all(dir<=0) && any(dir<0));
isTrivial = all(dir==0) || ...
(requireBothSides && oneSided) || ...
sum(trades) < minTrades || ...
tradePct > maxTradePct || ...
(requireBothSides && (longPct < minLongPct || shortPct <
minShortPct));
end
% ------------------------------------------------------------function [devTable, forecastCache] =
evaluateArmaConfigsOnDev( ...
logRet, simpRet, allowedModels, idx, cfg)
allRows = {};
forecastCache = containers.Map('KeyType','char','ValueType','any');
evalRaw = any(cfg.stratBModes == "raw");
evalVol = any(cfg.stratBModes == "volscaled");
evalPct = any(cfg.stratBModes == "percentile");
ddMults = 1;
if cfg.useDoubleDown
ddMults = [1, cfg.ddMultipliers];
end
nPerModel = (evalRaw*length(cfg.rawGrid) + evalVol*length(cfg.volGrid) + ...
evalPct*length(cfg.pctGrid)) * length(ddMults);
totalIter = size(allowedModels,1) * nPerModel;
iter = 0;
h = waitbar(0, 'Evaluating configs on development sample...');
for m = 1:size(allowedModels,1)
p = allowedModels(m,1); q = allowedModels(m,2);
[fc, condVol] = rollingArmaForecasts( ...
logRet, idx.devStart, idx.devEnd, cfg.winLen, p, q, cfg.refitEvery,
cfg.useGarch);
forecastCache(sprintf('%d_%d',p,q)) = struct('fc',fc,'condVol',condVol);
devRet = simpRet(idx.devStart:idx.devEnd);
for ddMult = ddMults
if evalRaw
for k = 1:length(cfg.rawGrid)
iter = iter+1; waitbar(iter/totalIter, h);
thr = cfg.rawGrid(k);
sigRaw = buildSignalFromMode(fc, condVol, "raw", thr,
cfg.pctLookback, ddMult);
if shouldApplyGarchScaling(cfg, "raw")
sigRaw = applyGarchScalingDD(sigRaw, condVol,
cfg.sigmaTarget);
end
sig = applyMinHoldingPeriod(sigRaw, cfg.minHold);
[sNet,isTrivial,flatPct,longPct,shortPct] =
evaluateSignal( ...
sig, devRet, cfg.tc, cfg.rfDaily, cfg.minTradesDev, ...
cfg.requireBothSides, cfg.minLongPct, cfg.minShortPct,
cfg.maxTradePct);
allRows(end+1,:) = {p,q,"raw",thr,ddMult, ...


sNet.CumReturn,sNet.AnnReturn,sNet.AnnVol,sNet.Sharpe, ...
sNet.NTrades,flatPct,longPct,shortPct,isTrivial}; %#ok<AGROW>
end
end
if evalVol
for k = 1:length(cfg.volGrid)
iter = iter+1; waitbar(iter/totalIter, h);
thr = cfg.volGrid(k);
sigRaw = buildSignalFromMode(fc, condVol, "volscaled", thr,
cfg.pctLookback, ddMult);
if shouldApplyGarchScaling(cfg, "volscaled")
sigRaw = applyGarchScalingDD(sigRaw, condVol,
cfg.sigmaTarget);
end
sig = applyMinHoldingPeriod(sigRaw, cfg.minHold);
[sNet,isTrivial,flatPct,longPct,shortPct] =
evaluateSignal( ...
sig, devRet, cfg.tc, cfg.rfDaily, cfg.minTradesDev, ...
cfg.requireBothSides, cfg.minLongPct, cfg.minShortPct,
cfg.maxTradePct);
allRows(end+1,:) = {p,q,"volscaled",thr,ddMult, ...
sNet.CumReturn,sNet.AnnReturn,sNet.AnnVol,sNet.Sharpe, ...
sNet.NTrades,flatPct,longPct,shortPct,isTrivial}; %#ok<AGROW>
end
end
if evalPct
if ddMult == 1
pctThr = precomputePercentileThresholds(abs(fc),
cfg.pctLookback, cfg.pctGrid);
for k = 1:length(cfg.pctGrid)
iter = iter+1; waitbar(iter/totalIter, h);
thr = cfg.pctGrid(k);
sigRaw = makeSignalsPercentilePrecomputed(fc,
pctThr(:,k));
if shouldApplyGarchScaling(cfg, "percentile")
```

```
sigRaw = applyGarchScalingDD(sigRaw, condVol,
cfg.sigmaTarget);
end
sig = applyMinHoldingPeriod(sigRaw, cfg.minHold);
[sNet,isTrivial,flatPct,longPct,shortPct] =
evaluateSignal( ...
sig, devRet, cfg.tc, cfg.rfDaily,
cfg.minTradesDev, ...
cfg.requireBothSides, cfg.minLongPct,
cfg.minShortPct, cfg.maxTradePct);
allRows(end+1,:) = {p,q,"percentile",thr,1, ...
sNet.CumReturn,sNet.AnnReturn,sNet.AnnVol,sNet.Sharpe, ...
sNet.NTrades,flatPct,longPct,shortPct,isTrivial}; %#ok<AGROW>
end
else
iter = iter + length(cfg.pctGrid);
waitbar(iter/totalIter, h);
end


end
end
end
close(h)
devTable = cell2table(allRows, 'VariableNames', ...
{'p','q','Mode','Threshold','DDMult','CumNet','AnnRetNet','AnnVolNet','Sharpe
Net', ...
'NTrades','FlatPct','LongPct','ShortPct','IsTrivial'});
end
% ------------------------------------------------------------function bestRow =
selectBestWithParsimony(nonTrivTable, parsimonyTol)
nonTrivTable = sortrows(nonTrivTable, 'SharpeNet', 'descend');
bestRow = nonTrivTable(1,:);
for i = 2:height(nonTrivTable)
cand = nonTrivTable(i,:);
candComplexity = cand.p + cand.q + (cand.DDMult - 1);
bestComplexity = bestRow.p + bestRow.q + (bestRow.DDMult - 1);
if candComplexity < bestComplexity && cand.SharpeNet >= bestRow.SharpeNet
- parsimonyTol
bestRow = cand; break
end
end
end
% ------------------------------------------------------------function bestPerModel =
bestConfigPerModel(devTable, validRows)
uq = unique(devTable(:,{'p','q'}), 'rows');
bestPerModel = table();
for i = 1:height(uq)
rows = validRows & devTable.p==uq.p(i) & devTable.q==uq.q(i);
if any(rows)
tmp = sortrows(devTable(rows,:), 'SharpeNet', 'descend');
bestPerModel = [bestPerModel; tmp(1,:)]; %#ok<AGROW>
end
end
bestPerModel = sortrows(bestPerModel, 'SharpeNet', 'descend');
end
% ------------------------------------------------------------% Used in Section 13 only — re-applies the (S, L,
B) selected in
% Section 12 to the ARMA test-sample indices so MA can sit alongside
% ARMA / ARMA-GARCH in the consolidated comparison.
function sig = buildMASignal(price, startIdx, endIdx, S, L, B)
nPrice = length(price);
shortMA = movmean(price, [S-1, 0]);
longMA = movmean(price, [L-1, 0]);
R = (shortMA - longMA) ./ longMA;
fullSig = zeros(nPrice, 1);
fullSig(R > B) = 1;
fullSig(R < -B) = -1;
fullSig(1:L-1) = 0;
sig = fullSig(startIdx:endIdx);
end
```

Task 3 Code

```
clc
%readtable into Matlab
timeseries
script\MRK_H&L.csv'])
y1=timeseries.Spread

=readtable(['C:\Users\ytshe\OneDrive\desktop\matlab

subplot(2,1,1)
plot(y1)
title('Time Series Plot of Spread');
xlabel('Time');
ylabel('Spread Value');
subplot(2,1,2)
hist(y1,50)
title('Time Series Spread Distribution')
xlabel('Spread Value');
ylabel('Numbers')
autocorr(y1,"NumLags",100)
% Full Sample AR(1)
EstMdl=arima(1,0,0);
EstMdl=estimate(EstMdl,log(y1));
[E,V]=infer(EstMdl,log(y1)); % Extract Residuals
autocorr(E,"NumLags",100);
plot(E)
title('Residuals of Log-AR(1)')
```

```
% Full Sample HAR
Y2=log(y1)
y1_daily = y2(23:end); % Dependent variable (starts later to fit lags)
y1_d_lag1 = y2(22:end-1);
y1_w = movmean(y2(1:end-1), [4 0]); % 5-day average
y1_w = y1_w(22:end);
y1_m = movmean(y2(1:end-1), [21 0]); % 22-day average
y1_m = y1_m(22:end);
harTable = table(y1_daily, y1_d_lag1, y1_w, y1_m);
mdl = fitlm(harTable, 'y1_daily ~ y1_d_lag1 + y1_w + y1_m');
disp(mdl)
Z3=mdl.Residuals.Standardized
r2_har = mdl.Rsquared.Adjusted
autocorr(Z3,'NumLags',100)
% Full sample ARMA
BICmat = zeros(6, 6); % Initialize BIC matrix for storing results
for p=0:5
for q=0:5
Mdl=arima(p,0,q)
Mdl=estimate(Mdl,log(y1))
Info=summarize(Mdl);


BIC=Info.BIC;
BICmat(p+1,q+1)=BIC;
end
end
[pind,qind]=find(BICmat == min(BICmat,[],'all'))
pstar = pind-1;
qstar=qind-1;
disp([pstar,qstar])
OptMdl=arima(pstar,0,qstar);
OptMdl=estimate(OptMdl,log(y1));
disp (OptMdl)
[E2,V]=infer(OptMdl,log(y1))
autocorr(E2,'NumLags',100)

%forecast HAR model (log)
y_log=log(y1)
IS=2000;
y_IS=y1(1:IS);
y_OS=y1(IS+1:end);
T = length(y_log);
n_forecasts = T - IS;
forecasts = NaN(n_forecasts, 1);
for t = 1:n_forecasts
% --- 1. Define the estimation window --window_start = t;
window_end
= t + IS - 1;
RV_window
= y_log(window_start:window_end);
% --- 2. Build HAR regressors within the window --% Minimum lags needed: 22 (monthly component)
n = length(RV_window);
% Daily, weekly (5-day avg), monthly (22-day avg)
RV_d = RV_window(22:n-1);
% daily lag
RV_w = movmean(RV_window, [4 0]); RV_w = RV_w(22:n-1); % 5-day avg
RV_m = movmean(RV_window, [21 0]); RV_m = RV_m(22:n-1); % 22-day avg
RV_y = RV_window(23:n);
% dependent var
% --- 3. OLS estimation --X = [ones(length(RV_d),1), RV_d, RV_w, RV_m];
beta = (X' * X) \ (X' * RV_y);
% OLS: [α, β_d, β_w, β_m]
% 4. In-window residual variance (σ²) for bias correction
resid
= RV_y - X * beta;
sigma2
= (resid' * resid) / (length(resid) - 4);
% --- 4. Build regressors for the forecast point --rv_d_fc = y_log(window_end);
% last daily value
rv_w_fc = mean(y_log(window_end-4 : window_end));
% last 5-day avg
rv_m_fc = mean(y_log(window_end-21: window_end));
% last 22-day avg
x_fc = [1, rv_d_fc, rv_w_fc, rv_m_fc];


% --- 5. One-step-ahead forecast --forecasts(t) = exp((x_fc * beta)+sigma2/2);
end
plot([forecasts y_OS])
legend('Forecast HAR','True Value')
MSPE = mean((forecasts-y_OS).^2);
QLIKE = mean((y_OS./forecasts)-log(y_OS./forecasts)-1);
output=array2table([MSPE;QLIKE],'VariableNames',{'log
HAR'},"RowNames",{'MSPE','QLIKE'})
Table=table(y_OS,forecasts)
mdl_har=fitlm(Table,'y_OS ~ forecasts')
disp(mdl_har)
r2_har=mdl_har.Rsquared.Adjusted

% Mincer-Zarnowitz Regresson

%Wald test
% Step 1: Run OLS regression
X = [ones(length(y_OS),1), forecasts];
% design matrix
b = X \ y_OS;
% OLS estimates [alpha; beta]
% Step 2: Compute residuals and covariance matrix
resid = y_OS - X*b;
sigma2 = (resid'*resid) / (length(y_OS) - 2);
V = sigma2 * inv(X'*X);
% covariance of estimates
```

```
% Step 3: Define restriction: alpha=0, beta=1
R = eye(2);
% restriction matrix
r = [0; 1];
% hypothesized values
% Step 4: Compute Wald statistic
diff = R*b - r;
W = diff' / (R * V * R') * diff;
% Step 5: p-value (chi-squared with 2 df)
pValue = 1 - chi2cdf(W, 2);
disp([W,pValue])

%Forecast AR(1) model (log)
EstMdl=arima(1,0,0);
yf=zeros(length(y_OS),1);
EstMdl=estimate(EstMdl,log(y_IS))
[E,V]=infer(EstMdl,log(y_IS))
for t = 1:length(y_OS)
y_EW = y_log(t:IS+t-1);%rolling window
yf(t,1) = exp(forecast(EstMdl,1,y_EW)+V(1)/2);
end
plot([yf y_OS]);
legend('Forecast AR(1)','True Value')


MSPE = mean((yf-y_OS).^2);
QLIKE = mean((y_OS./yf)-log(y_OS./yf)-1);
output=array2table([MSPE;QLIKE],'VariableNames',{'AR(1)'},"RowNames",{'MSPE',
'QLIKE'})
Table=table(y_OS,yf)
% Mincer-Zarnowitz Regresson
mdl_ar1=fitlm(Table,'y_OS ~ yf')
disp(mdl_ar1)
r2_ar1=mdl_ar1.Rsquared.Adjusted
%Wald test
% Step 1: Run OLS regression
X = [ones(length(y_OS),1), yf];
% design matrix
b = X \ y_OS;
% OLS estimates [alpha; beta]
% Step 2: Compute residuals and covariance matrix
resid = y_OS - X*b;
sigma2 = (resid'*resid) / (length(y_OS) - 2);
V = sigma2 * inv(X'*X);
% covariance of estimates
% Step 3: Define restriction: alpha=0, beta=1
R = eye(2);
% restriction matrix
r = [0; 1];
% hypothesized values
% Step 4: Compute Wald statistic
diff = R*b - r;
W = diff' / (R * V * R') * diff;
% Step 5: p-value (chi-squared with 2 df)
pValue = 1 - chi2cdf(W, 2);
disp([W,pValue])
%ARMA estimation and forecast (log)
BICmat = zeros(6, 6); % Initialize BIC matrix for storing results
for p=0:5
for q=0:5
Mdl=arima(p,0,q)
Mdl=estimate(Mdl,log(y_IS))
Info=summarize(Mdl);
BIC=Info.BIC;
BICmat(p+1,q+1)=BIC;
end
end
[pind,qind]=find(BICmat == min(BICmat,[],'all'))
pstar = pind-1;
qstar=qind-1;
disp([pstar,qstar])
OptMdl=arima(pstar,0,qstar);
OptMdl=estimate(OptMdl,log(y_IS));
disp (OptMdl)
[E V]=infer(OptMdl,log(y_IS))
Yf=zeros(length(y_OS),1);
for t=1:length(Yf)


Y_EW=y_log(t:IS+t-1); %rolling window
Yf(t) = exp(forecast(OptMdl, 1, 'Y0', Y_EW)+V(1)/2);
end
plot([Yf y_OS])
MSPE = mean((Yf-y_OS).^2);
QLIKE = mean((y_OS./Yf)-log(y_OS./Yf)-1)
output=array2table([MSPE;QLIKE],'VariableNames',{'ARMA(1,2)'},"RowNames",{'MS
PE','QLIKE'})
Table=table(y_OS,Yf)
mdl_arma=fitlm(Table,'y_OS ~ Yf') % Mincer-Zarnowitz Regresson
disp(mdl_arma)
r2_arma=mdl_arma.Rsquared.Adjusted
%Wald test
% Step 1: Run OLS regression
X = [ones(length(y_OS),1), Yf];
% design matrix
b = X \ y_OS;
% OLS estimates [alpha; beta]
% Step 2: Compute residuals and covariance matrix
resid = y_OS - X*b;
sigma2 = (resid'*resid) / (length(y_OS) - 2);
V = sigma2 * inv(X'*X);
% covariance of estimates
% Step 3: Define restriction: alpha=0, beta=1
```

```
R = eye(2);
% restriction matrix
r = [0; 1];
% hypothesized values
% Step 4: Compute Wald statistic
diff = R*b - r;
W = diff' / (R * V * R') * diff;
% Step 5: p-value (chi-squared with 2 df)
pValue = 1 - chi2cdf(W, 2);
disp([W,pValue])

% Compare three
output=array2table([r2_har;r2_arma;r2_ar1],'VariableNames',{'Mincer-Zarnowitz
Rsquared'},'RowNames', ...
{'HAR','ARMA(1,2)','AR(1)'})
%DM test
%ARMA VS HAR (MSPE)
e1 = y_OS - Yf; % errors from ARMA
e2 = y_OS - forecasts; % errors from HAR
d = e1.^2 - e2.^2;
dBar = mean(d);
T = length(d);
autoCov = xcov(d, 'biased');
centerIdx = ceil(length(autoCov)/2);
varD = (1/T) * autoCov(centerIdx); % zero-lag autocovariance


DM_a1 = dBar / sqrt(varD)
pValue_a1 = 2 * (1 - normcdf(abs(DM_a1)));
a1 = [DM_a1 pValue_a1]
%QLIKE
L1 = y_OS./Yf - log(y_OS./Yf) - 1; %ARMA
L2 = y_OS./forecasts - log(y_OS./forecasts) - 1
d = L1 - L2;

%HAR

dBar = mean(d);
T = length(d);
autoCov = xcov(d, 'biased');
centerIdx = ceil(length(autoCov)/2);
varD = (1/T) * autoCov(centerIdx);
DM_a2 = dBar / sqrt(varD);
pValue_a2 = 2 * (1 - normcdf(abs(DM_a2)));
a2=[DM_a2 pValue_a2]
%AR VS HAR
e1 = y_OS - yf; % errors from AR
e2 = y_OS - forecasts; % errors from HAR
d = e1.^2 - e2.^2;
dBar = mean(d);
T = length(d);
autoCov = xcov(d, 'biased');
centerIdx = ceil(length(autoCov)/2);
varD = (1/T) * autoCov(centerIdx); % zero-lag autocovariance
DM_b1 = dBar / sqrt(varD)
pValue_b1 = 2 * (1 - normcdf(abs(DM_b1)));
b1=[DM_b1 pValue_b1 ]
%QLIKE
L1 = y_OS./yf - log(y_OS./yf) - 1; %AR
L2 = y_OS./forecasts - log(y_OS./forecasts) - 1
d = L1 - L2;

%HAR

dBar = mean(d);
T = length(d);
autoCov = xcov(d, 'biased');
centerIdx = ceil(length(autoCov)/2);
varD = (1/T) * autoCov(centerIdx);
DM_b2 = dBar / sqrt(varD);
pValue_b2 = 2 * (1 - normcdf(abs(DM_b2)));
b2 = [DM_b2 pValue_b2]
%AR VS ARMA
e1 = y_OS - yf; % errors from AR
e2 = y_OS - Yf; % errors from ARMA
d = e1.^2 - e2.^2;
dBar = mean(d);
T = length(d);
autoCov = xcov(d, 'biased');
centerIdx = ceil(length(autoCov)/2);
varD = (1/T) * autoCov(centerIdx); % zero-lag autocovariance


DM_c1 = dBar / sqrt(varD)
pValue_c1 = 2 * (1 - normcdf(abs(DM_c1)));
c1 = [DM_c1 pValue_c1]
%QLIKE
L1 = y_OS./yf - log(y_OS./yf) - 1; %AR
L2 = y_OS./Yf - log(y_OS./Yf) - 1 %ARMA
d = L1 - L2;
dBar = mean(d);
T = length(d);
autoCov = xcov(d, 'biased');
centerIdx = ceil(length(autoCov)/2);
varD = (1/T) * autoCov(centerIdx);
DM_c2 = dBar / sqrt(varD);
pValue_c2 = 2 * (1 - normcdf(abs(DM_c2)));
c2 = [DM_c2 pValue_c2]
output=array2table([a1;b1;c1],'VariableNames',{'DM','pValue'},'RowNames',{'AR
MA vs HAR','AR vs HAR','AR vs ARMA'})
output=array2table([a2;b2;c2],'VariableNames',{'DM','pValue'},'RowNames',{'AR
MA vs HAR','AR vs HAR','AR vs ARMA'})
```